테토리스

제7회 우석 컴퓨팅사고 창작대회

WCMS

(Woosuk Computer Management System)

2025.11.22

김영빈

목 차

1 작품 소개

작품 개요

DATABASE

2 주요기능 소개

화면 구성

실행 로직

시연 동영상

3 실행 방법

환경 설정(Server)

환경 설정(Client)

4 향후 계획

향후 계획

5 참고자료

1. 작품 소개

작품 소개

분야: 전공융합

개발언어: 파이썬

개발동기

정보보안학과 전공생이자 근로장학생으로 교내 실습실을 관리하며, 현재의 수동적인 PC 점검 및 종료 방식이 자원 관리 측면에서 비효율적이라는 문제의식을 갖게 되었습니다.

특히 정보보안 전공자로서, 이러한 반복적인 프로세스를 파이썬을 통해 분석하고 자동화할 필요성을 체감했습니다.

또한 외국인 학생들이 PC에 외국어를 설치하면서, 한국학생들이 쓰는 프로그램이 외국어로 변경되는 불편함을 겪고 있었습니다.

작품개요

이 작품은 flask web framwork를 사용하여 만든 웹 서버에, 각 PC의 하드웨어/소프트웨어 자원 정보를 REST API를 통해 실시간으로 수집하며, 관리자가 웹 대시보드를 통해 원격으로 전원 제어, 계정 관리, 소프트웨어 배포 등의 명령을 내릴 수 있는 통합 관리 시스템입니다.

실습실 PC가 켜지면 자동으로 백그라운드 서비스가 서버에 연결되며, 관리자는 웹 대시보드에 접속하여 클릭 한 번으로 전체 PC의 상태를 확인하고 제어할 수 있습니다.

작품 소개

분야: 전공융합

개발언어: 파이썬

기존 방식

- 관리자가 직접 이동하여 육안 점검
- 문제 발생 시 현장 조치 필요
- PC 종료 여부 확인에 많은 시간 소요
- 외국인 학생들이 PC 언어 설정을 변경하여 한국 학생들이 불편을 겪음

WCMS

- 웹 대시보드에서 전체 PC 상태 한눈에 파악 (시각화)
- 클릭 한 번으로 일괄 전원 종료 및 관리 (자동화)
- 비정상 프로세스 및 하드웨어 상태 실시간 감지 (모니터링)
- 관리자가 클릭 한 번으로 한국계정과 유학생용 계정을 원격으로 생성 및 삭제하여, 매 수업마다 깨끗한 OS 환경을 제공

작품 소개

Server

- 언어: python3.13 이상
- 웹 프레임워크: Flask 3
- 템플릿 엔진: Jinja2
- 데이터베이스: SQLite3
- 인증: Flask-Session
- 보안: bcrypt (비밀번호 해싱)
- 배포: Gunicorn + Apache2
- 서버 환경: Linux/Ubuntu 22.04 LTS (동아리 서버, 내부망)

Client

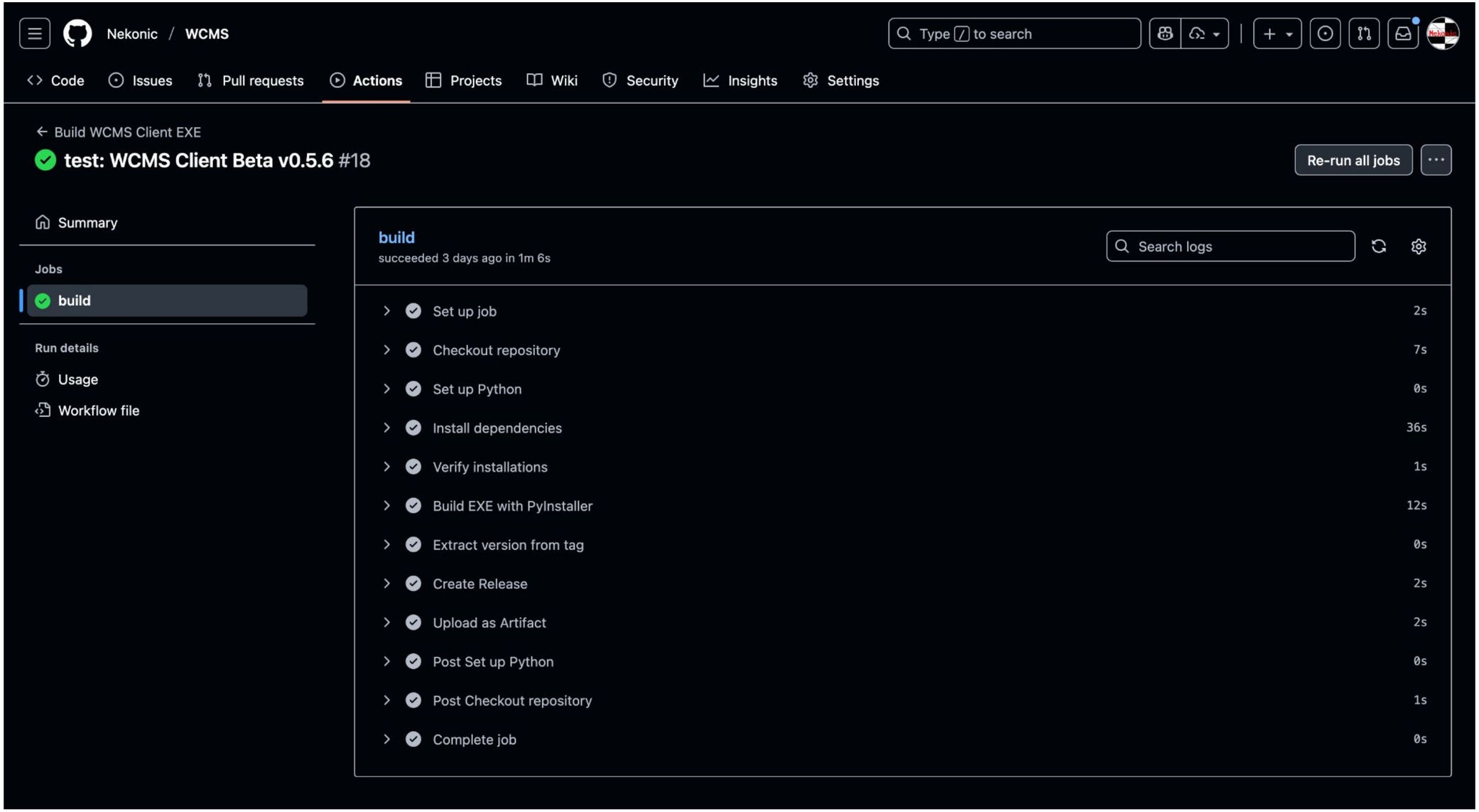
- 언어: python3.9 이상
- 시스템 정보 수집: psutil, wmi, platform
- Windows 서비스: pywin32
- HTTP API 통신: requests
- 빌드 도구: PyInstaller
- CI/CD: GitHub Actions (자동 빌드 및 Release)
- 실행 환경: Windows 10/11 백그라운드 서비스

작품 소개

코드 관리

- GIT & GITHUB를 통해 버전관리를 하고있습니다.
  https://github.com/Nekonic/WCMS
- 소스코드는 MIT 라이센스가 적용됩니다.
- 현재 기준 최신버전은 다음과 같습니다.
  Pre-release: WCMS Client v0.5.6
- GITHUB Actions를 사용하여 client 디렉토리 내의 코드를 .exe 파일로 빌드하여 tag를 생성하면 자동으로 빌드됩니다.

```
WCMS/
├── server/          # Flask 서버
│   ├── app.py       # 메인 애플리케이션
│   ├── templates/   # HTML 템플릿
│   └── migrations/  # DB 스키마
├── client/          # 클라이언트 프로그램
│   ├── main.py      # 메인 로직
│   ├── service.py   # Windows 서비스
│   ├── collector.py # 시스템 정보 수집
│   └── executor.py  # 명령 실행
├── test_all.py      # 통합 테스트
├── GUIDE.md         # 통합 가이드
└── STATUS.md        # 프로젝트 상태
```



GITHUB Actions로 코드를 exe파일로 빌드하는 과정

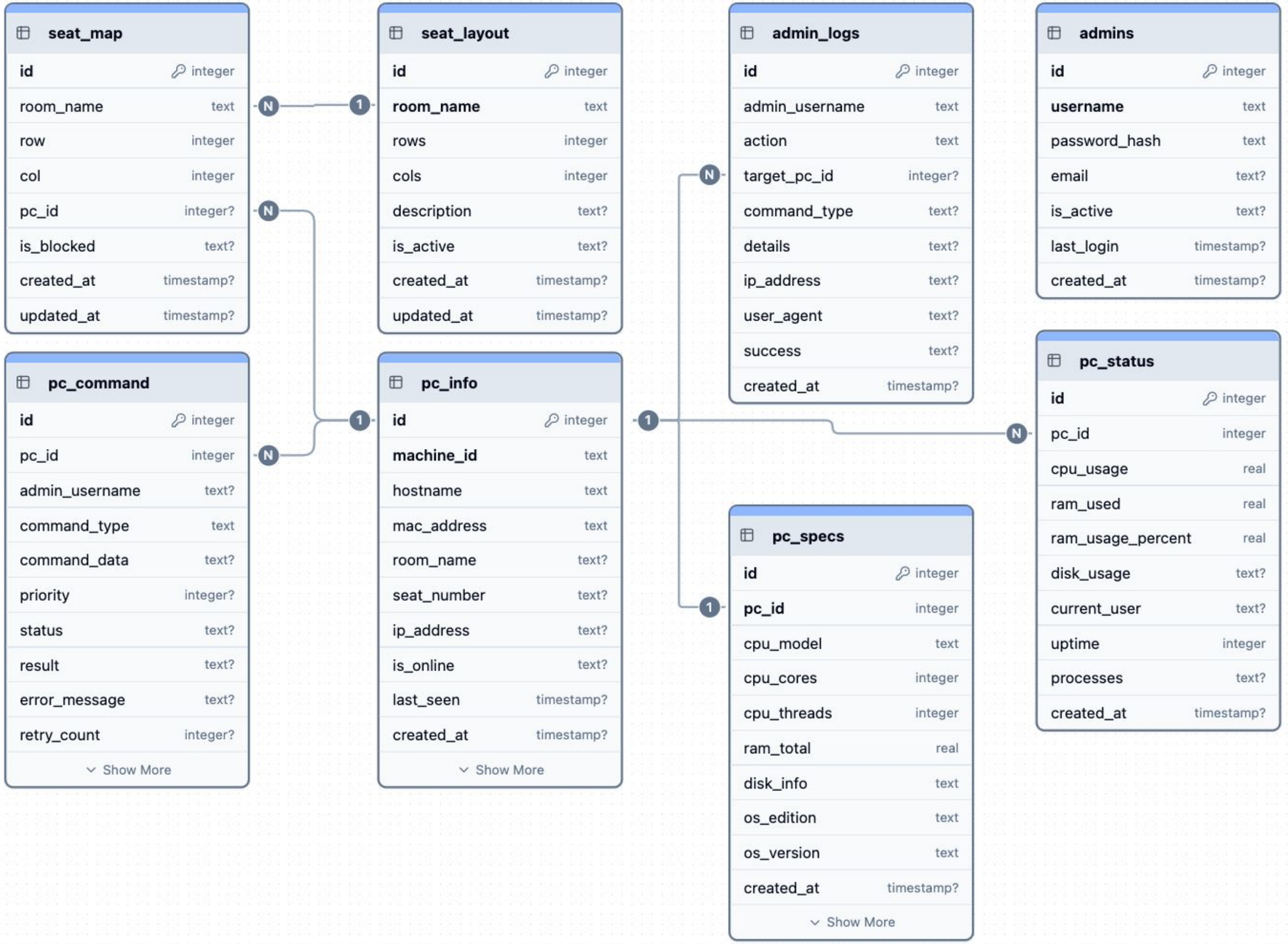
작품 소개



GITHUB Actions빌드가 정상적으로 끝나면, https://github.com/Nekonic/WCMS/releases

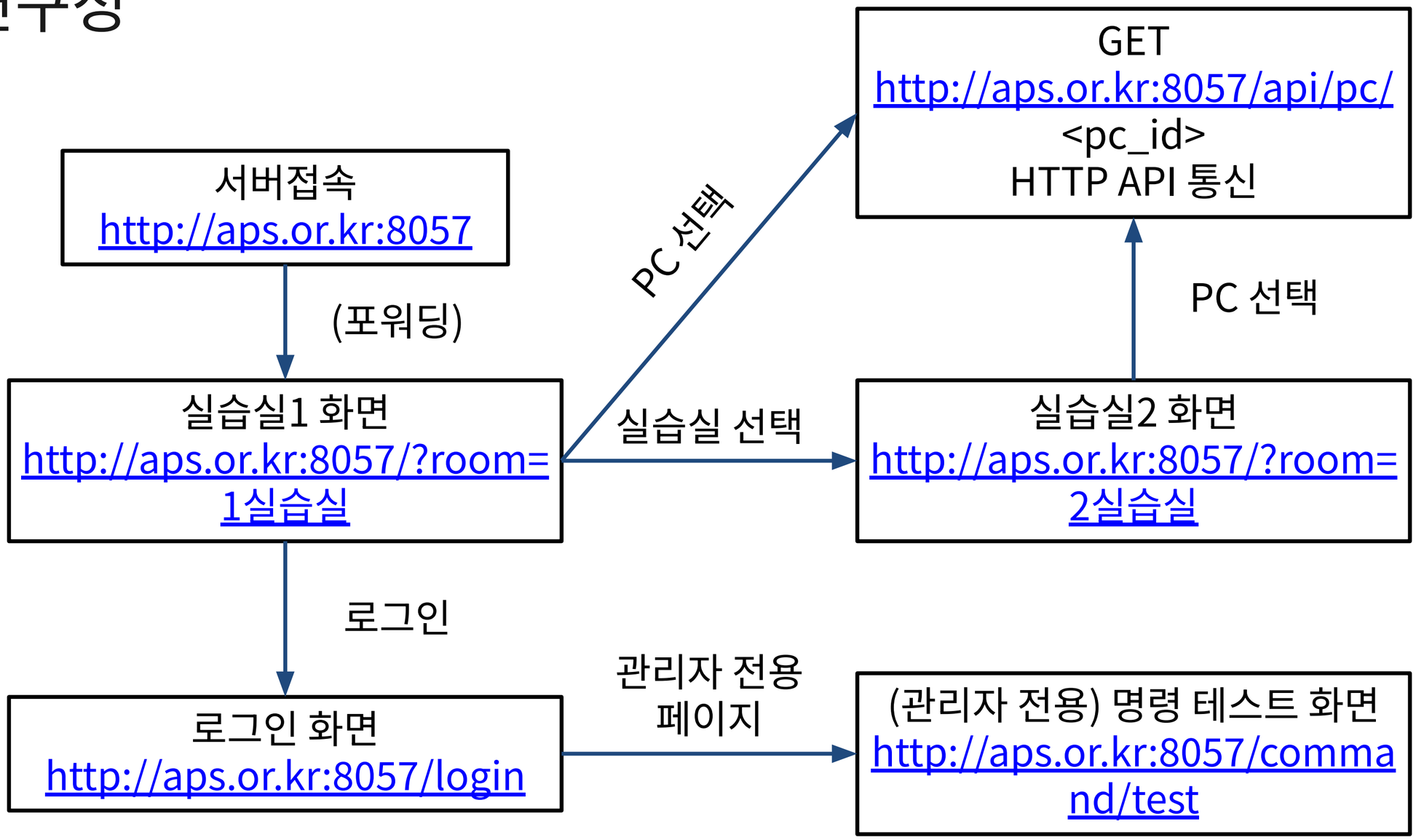
이 경로에 Assets를 눌러보면, 소스코드와 함께 WCMS-Client.exe 파일이 정상적으로 빌드된 것을 확인할 수 있습니다.

DATABASE


seat_map
id integer
room_name text
row integer
col integer
pc_id integer?
is_blocked text?
created_at timestamp?
updated_at timestamp?
seat_layout
id integer
room_name text
rows integer
cols integer
description text?
is_active text?
created_at timestamp?
updated_at timestamp?
admin_logs
id integer
admin_username text
action text
target_pc_id integer?
command_type text?
details text?
ip_address text?
user_agent text?
success text?
created_at timestamp?
admins
id integer
username text
password_hash text
email text?
is_active text?
last_login timestamp?
created_at timestamp?
pc_command
id integer
pc_id integer
admin_username text?
command_type text
command_data text?
priority integer?
status text?
result text?
error_message text?
retry_count integer?
Show More
pc_info
id integer
machine_id text
hostname text
mac_address text
room_name text?
seat_number text?
ip_address text?
is_online text?
last_seen timestamp?
created_at timestamp?
Show More
pc_status
id integer
pc_id integer
cpu_usage real
ram_used real
ram_usage_percent real
disk_usage text?
current_user text?
uptime integer
processes text?
created_at timestamp?
pc_specs
id integer
pc_id integer
cpu_model text
cpu_cores integer
cpu_threads integer
ram_total real
disk_info text
os_edition text
os_version text
created_at timestamp?
Show More
N
1

2. 주요기능 소개

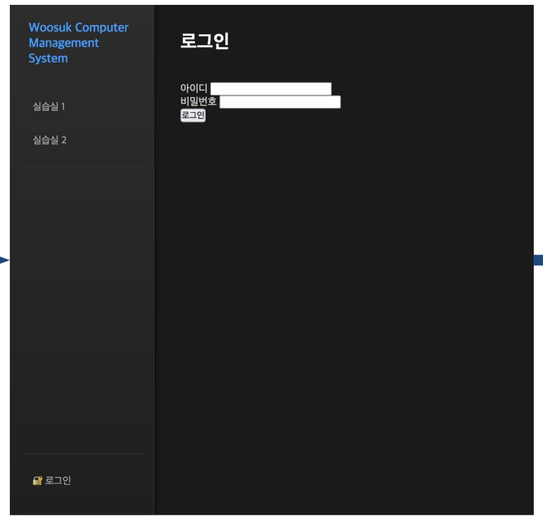
화면구성


서버접속
http://aps.or.kr:8057
(포워딩)
실습실1 화면
http://aps.or.kr:8057/?room=1실습실
PC 선택
GET
http://aps.or.kr:8057/api/pc/<pc_id>
HTTP API 통신
실습실 선택
실습실2 화면
http://aps.or.kr:8057/?room=2실습실
PC 선택
로그인
로그인 화면
http://aps.or.kr:8057/login
관리자 전용 페이지
(관리자 전용) 명령 테스트 화면
http://aps.or.kr:8057/command/test

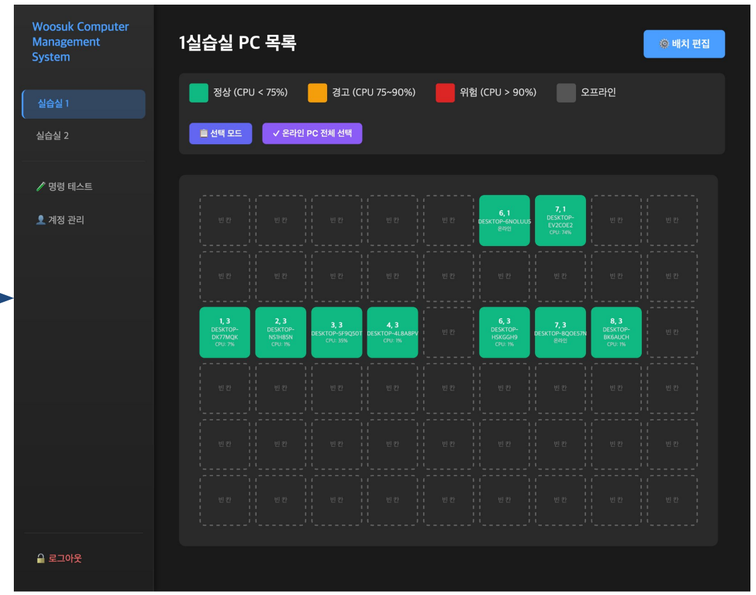
화면구성



로그인



로그인 후



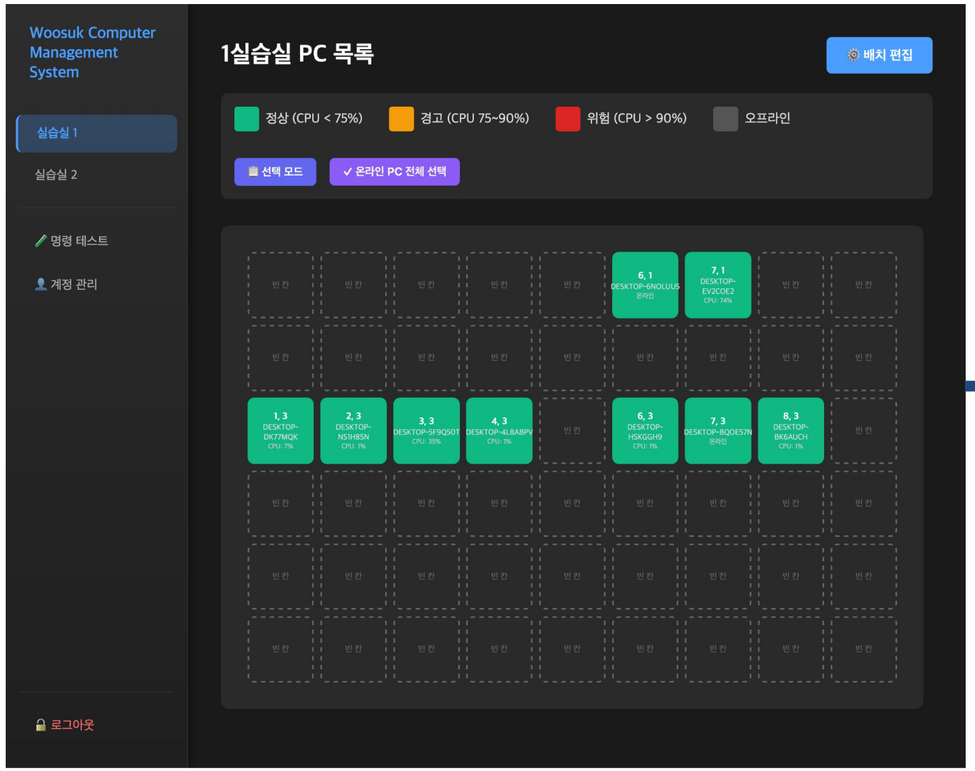
http://aps.or.kr:8057

http://aps.or.kr:8057?room=1실습실
(포워딩)

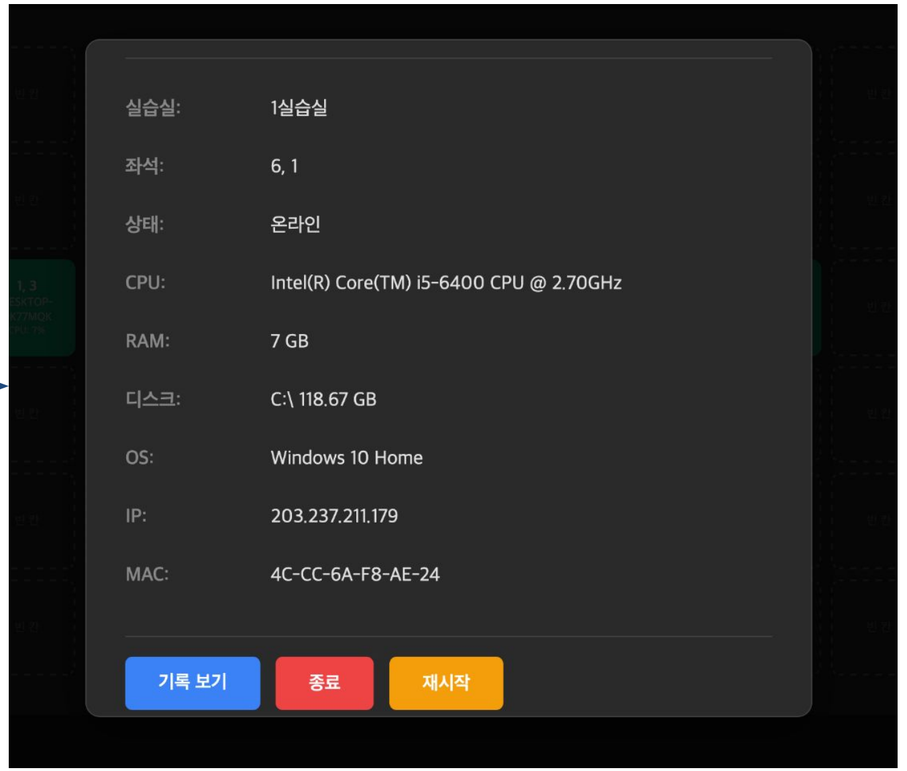
http://aps.or.kr:8057/login

http://aps.or.kr:8057/?room=1실습실
(리다이렉션)

화면구성



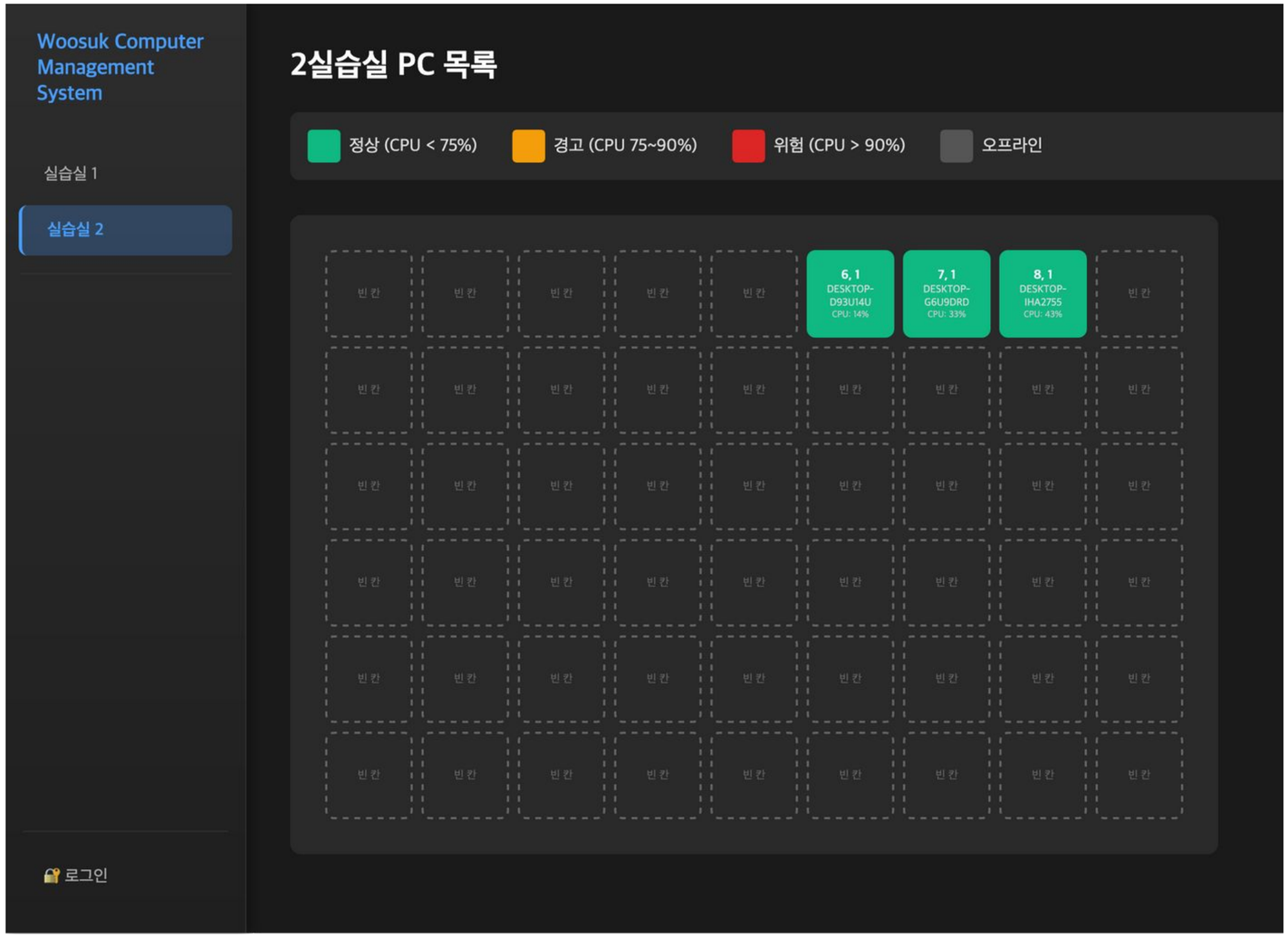
pc 클릭



http://aps.or.kr:8057/?room=1실습실

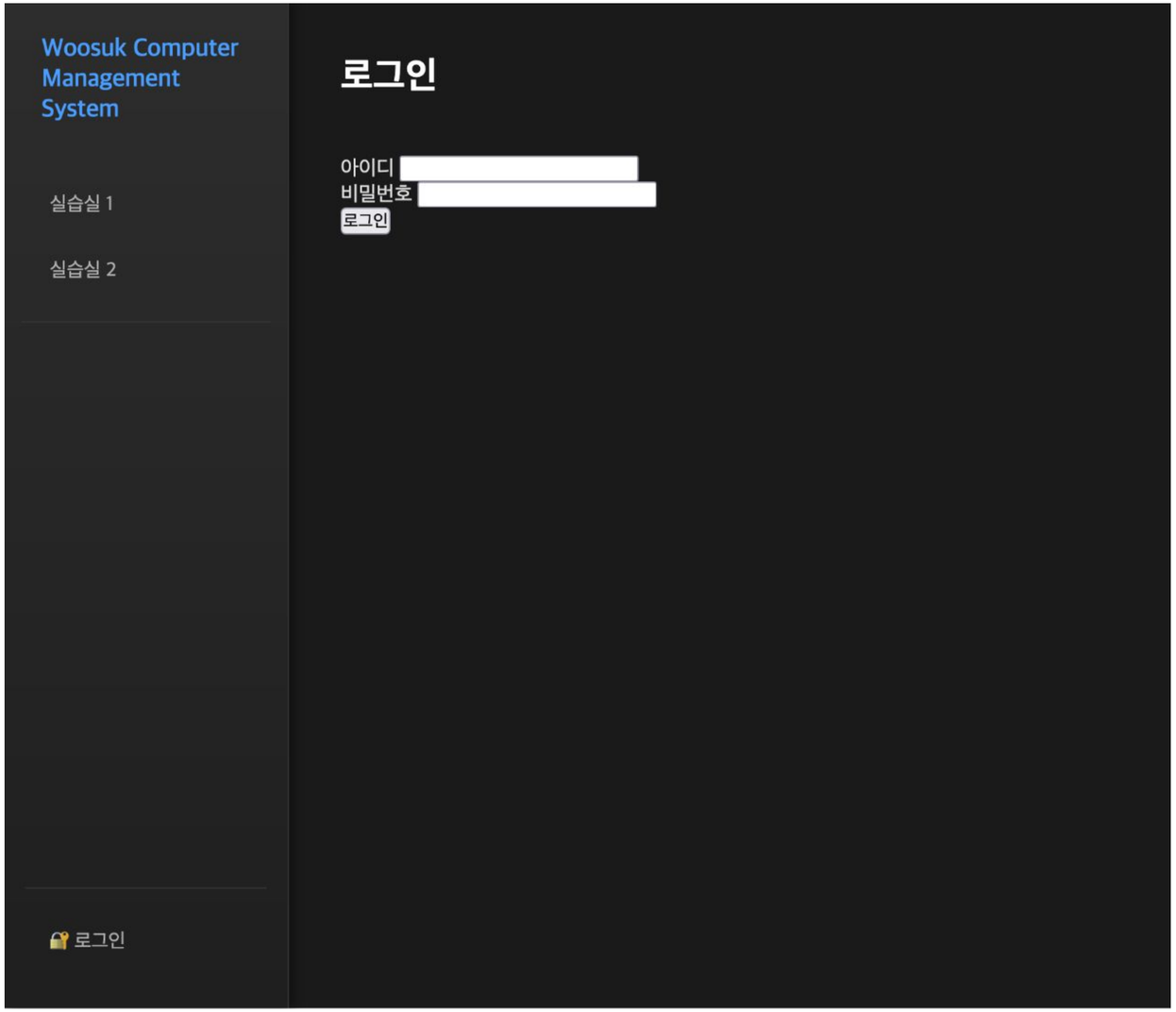
GET
http://aps.or.kr:8057/api/pc/<pc_id>
HTTP API 통신

화면구성



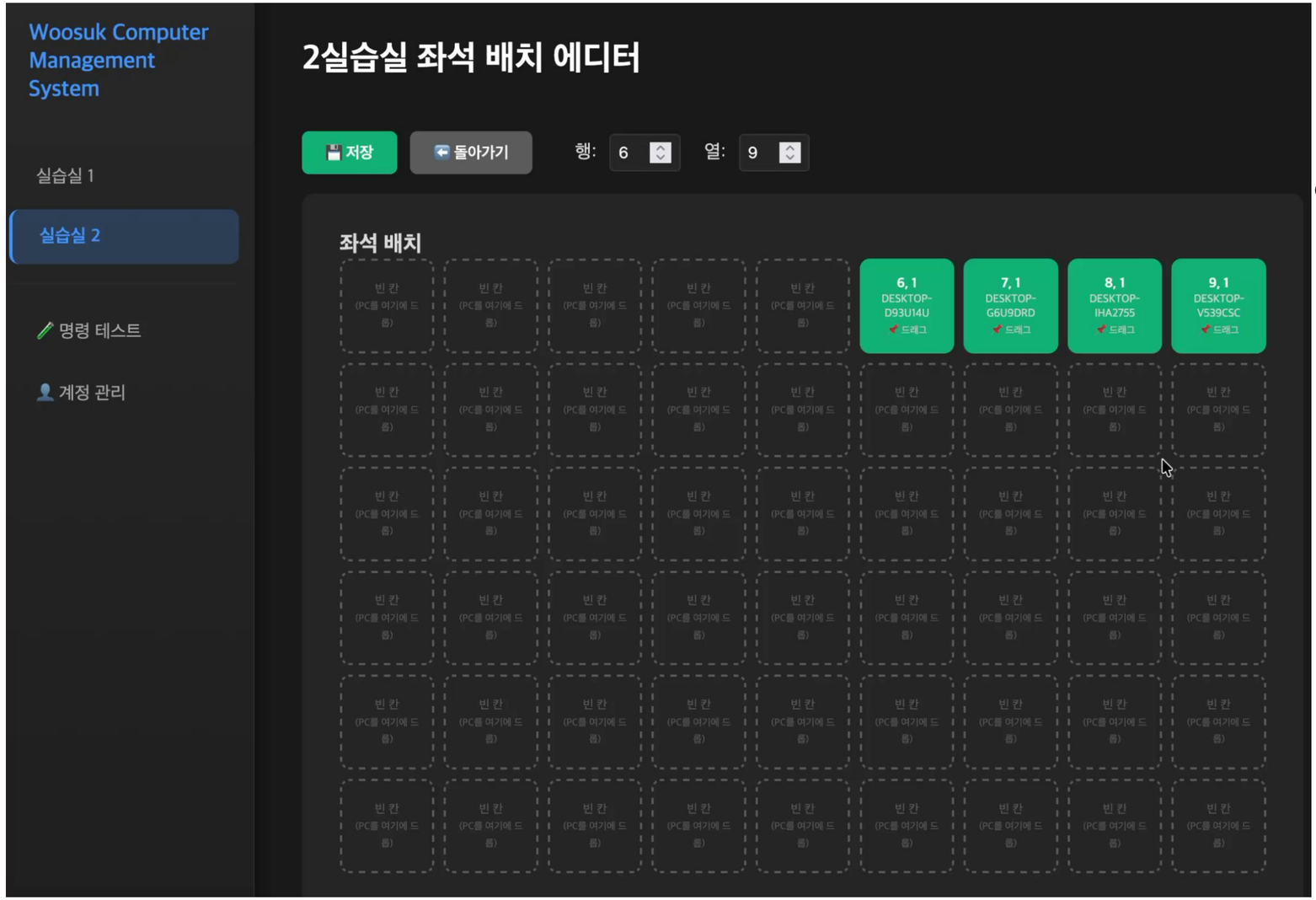
- 실시간 상태 시각화
  - PC의 온라인/오프라인 상태와 CPU 부하량(정상/경고/위험)을 색상으로 구분하여 직관적으로 모니터링합니다.
  - HTTP Long-polling 방식을 적용하여 새로고침 없이 실시간으로 상태가 업데이트됩니다.
  - 로그인을 하지 않은 상태에서는 배치편집, 선택모드, 온라인PC 전체 선택 등의 버튼은 보이지 않게 됩니다.

화면구성



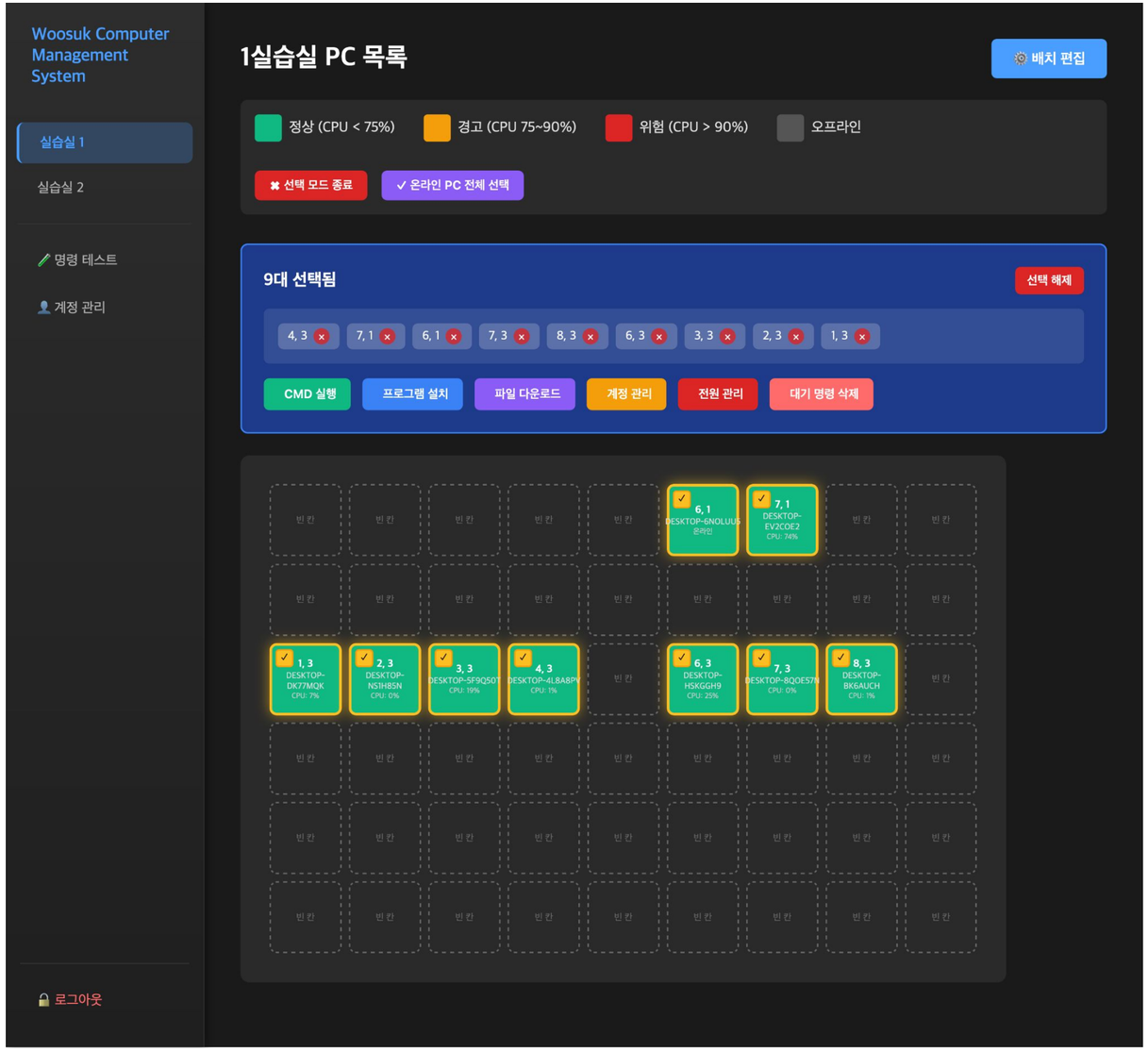
- 로그인 화면
  - 비밀번호는 파이썬의 bcrypt 라이브러리를 통해서 Hash 값으로 변환 후 DB에 저장된 Hash 값과 비교하여 로그인을 할 수 있습니다.

화면구성

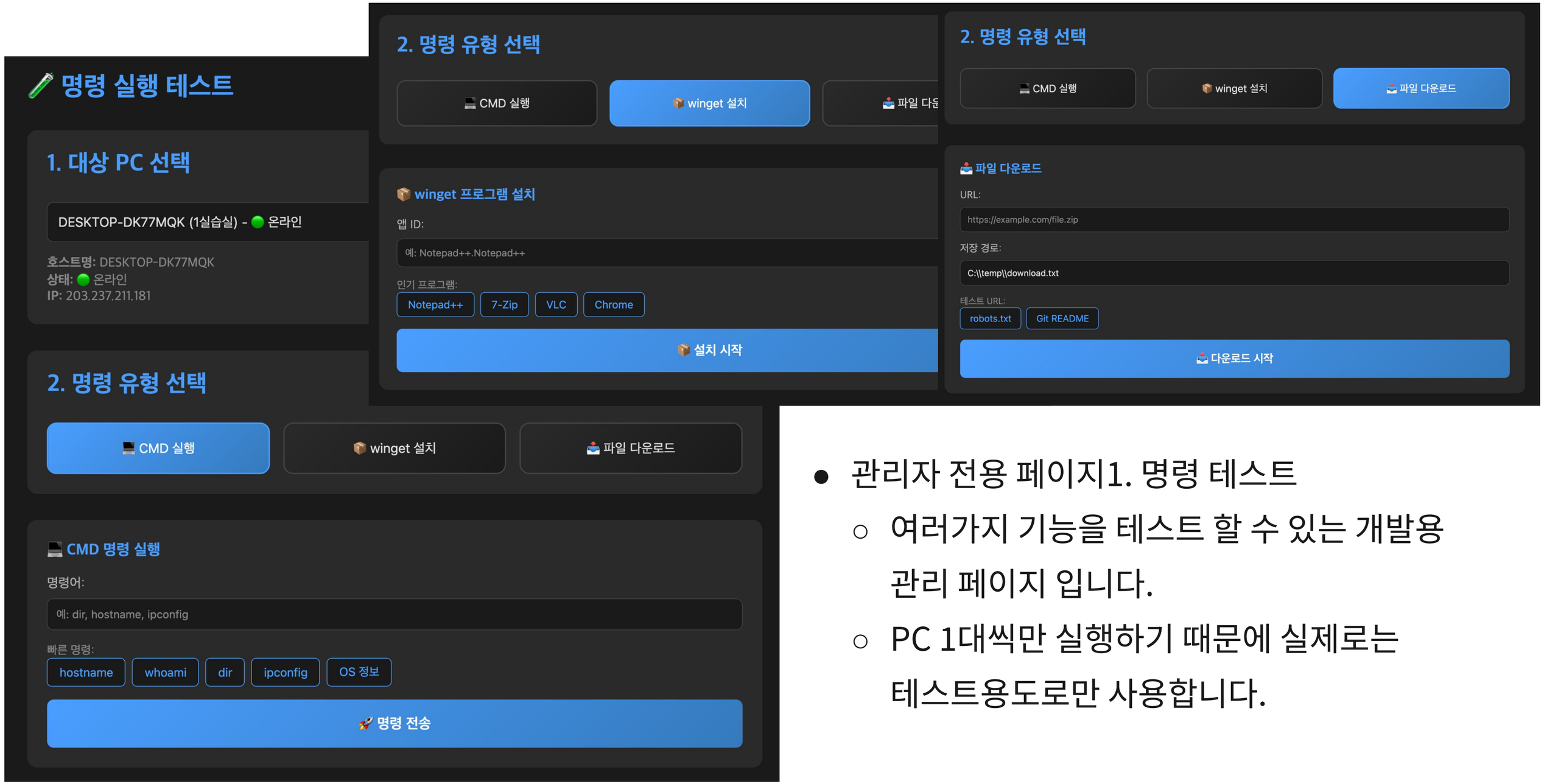


- 좌석 배치 에디터
  - 드래그 앤 드롭으로 편리하게 pc의 위치를 조절 할 수 있습니다.
  - pc를 교체하거나 새로운 좌석과 pc를 추가할 경우에도 편리하게 관리 할 수 있습니다.

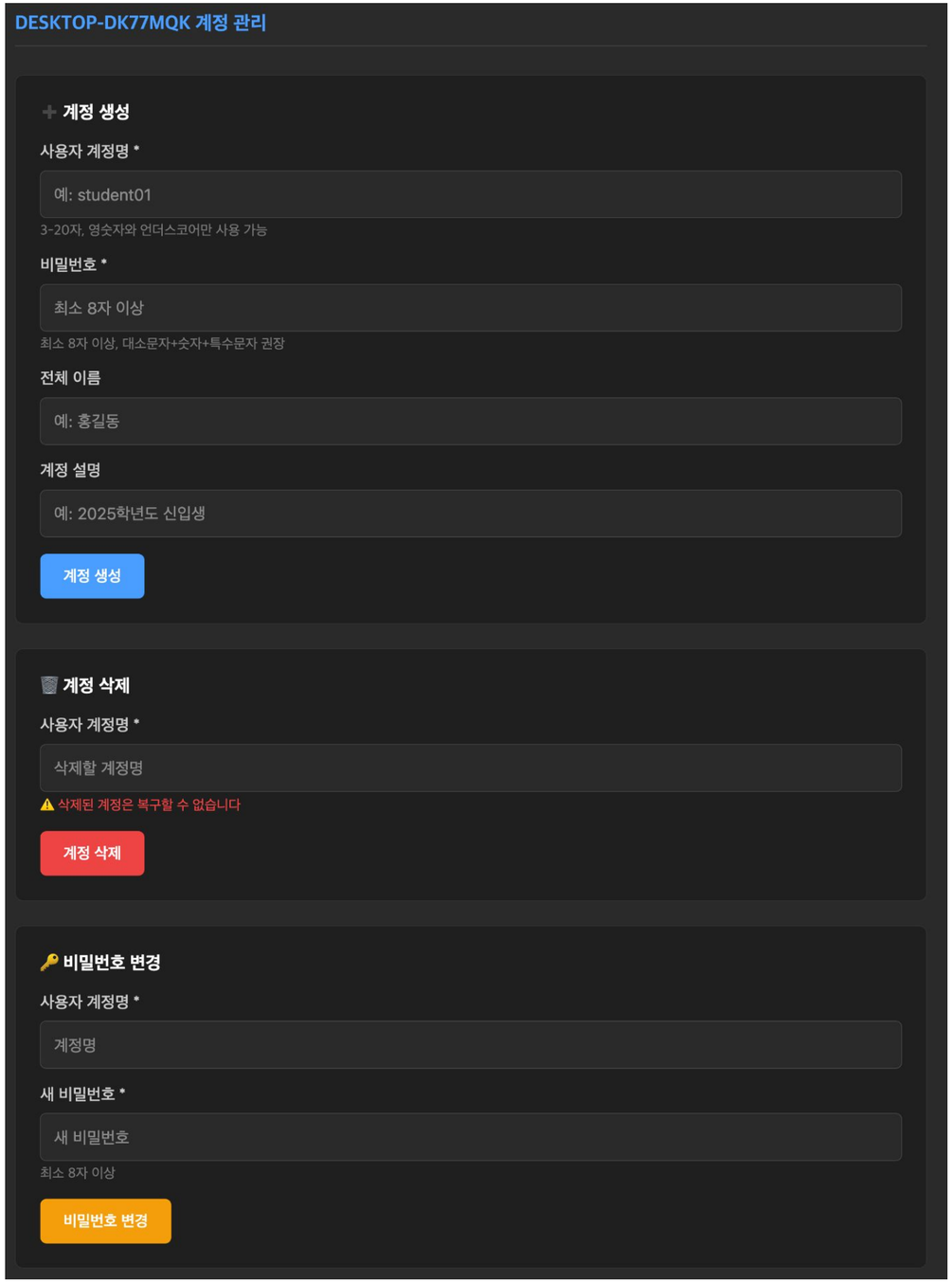
화면구성

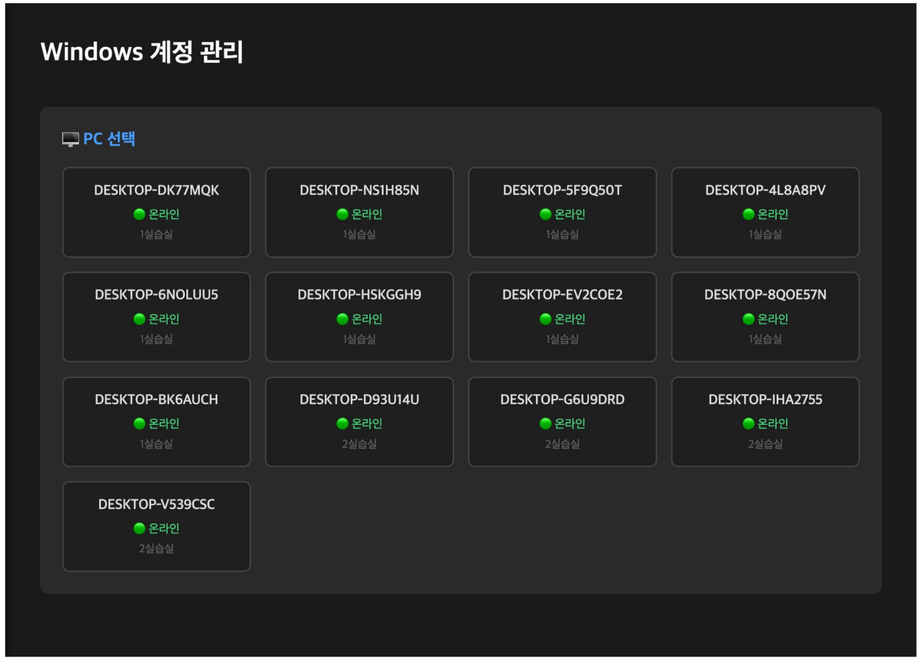


- 선택모드 & 온라인 PC 전체 선택
  - 로그인한 관리자는 PC를 선택하여 일괄 명령을 내릴 수 있습니다.
  - 프로그램 설치는 winget install 명령을 수행합니다.
  - CMD 실행을 누르면 DOS 명령을 내릴 수 있습니다. (dir, ipconfig 등등)
  - 파일 다운로드는 request 라이브러리를 통해서 클라이언트의 “C:\\temp\\” 경로에 다운로드 받을 수 있습니다.



- 관리자 전용 페이지1. 명령 테스트
  - 여러가지 기능을 테스트 할 수 있는 개발용 관리 페이지 입니다.
  - PC 1대씩만 실행하기 때문에 실제로는 테스트용도로만 사용합니다.





- 관리자 전용 페이지2. 계정관리
  - 윈도우 계정관리를 테스트 할 수 있는 페이지입니다.
  - 이 페이지 또한 PC 1대씩만 실행하기 때문에 실제로는 테스트용도로만 사용합니다.
  - 가끔씩 계정이 따로 만들어져있는 PC에 대해서 특정계정 삭제를 위해서 사용할수도 있습니다.

실행 로직

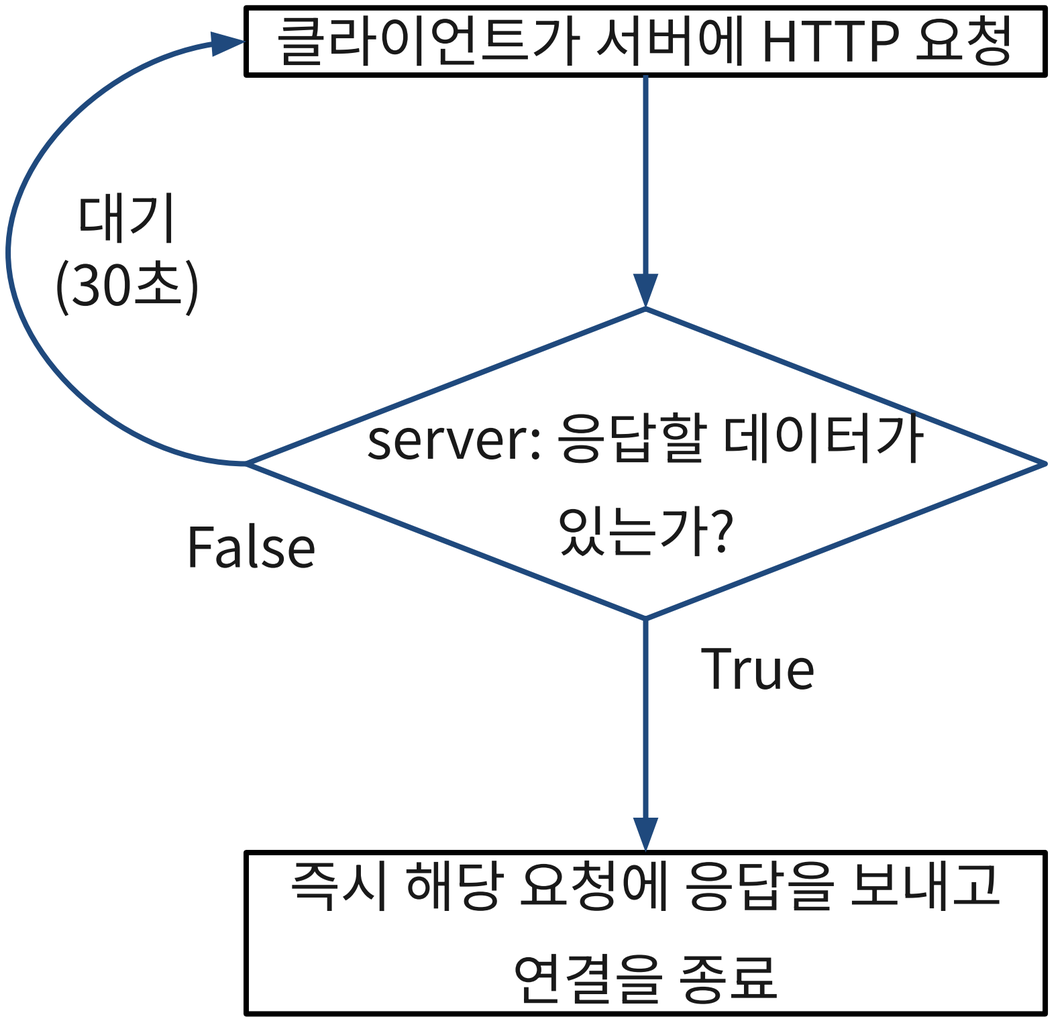
Client 주요 소스 코드 - 원격 명령 수신 (Long-polling)

```
def poll_command(stop_event):
    while not stop_event.is_set():
        # 1. 서버에 대기 중인 명령 확인 (30초 대기)
        r = requests.get(f"{SERVER_URL}api/client/command",
                         params={"machine_id": MACHINE_ID, "timeout": 30})

        if r.status_code == 200:
            cmd_data = r.json()
            # 2. 명령이 존재하면 실행
            if cmd_data.get('command_type'):
                cmd_type = cmd_data['command_type']
                # ... (중략) ...
                # 3. 명령 실행 및 결과 보고
                result = CommandExecutor.execute_command(cmd_type,
cmd_params)
                send_command_result(cmd_id, 'completed', result)
```

- 사용 파일: client/main.py
  1. 실시간 통신: requests 라이브러리를 사용하여 서버와 지속적으로 통신
  2. 부하 감소: 30초 타임아웃의 Long-polling 방식을 사용하여 네트워크 부하 최소화
  3. 결과 보고: 명령 실행 후 성공/실패 여부를 서버에 즉시 보고하여 관리자가 확인 가능

Long-polling 방식?



- 클라이언트가 서버로 HTTP 요청(주로 GET)을 보낸 뒤, 서버는 응답을 바로 보내지 않고 새로운 이벤트나 데이터가 생길 때까지 연결을 오래 잡고 있는다.
- 새 데이터가 생기면 서버가 그때 응답을 보내고 연결을 끊으며, 클라이언트는 응답을 받자마자 다시 요청을 보내 같은 과정을 반복한다.

Client 주요 소스 코드 - 윈도우 계정 관리 자동화

```
@staticmethod
def manage_account(action, username, password=None):
    try:
        if action == 'create':
            # 1. net user 명령어로 계정 생성
            cmd = f'net user "{username}" "{password}" /add'
            result = subprocess.run(cmd, shell=True, capture_output=True)
            return f"✅ 계정 생성 완료: {username}"

        elif action == 'delete':
            # 2. 계정 삭제 (환경 초기화)
            cmd = f'net user "{username}" /delete'
            result = subprocess.run(cmd, shell=True, capture_output=True)
            return f"✅ 계정 삭제 완료: {username}"
    except Exception as e:
        return f"❌ 관리 실패: {str(e)}"
```

- 사용 파일: client/executor.py

1. 자동화: subprocess 모듈을 활용해 CMD 명령어를 파이썬으로 제어
2. 환경 초기화: 클릭 한 번으로 계정을 삭제/생성하여, 이전 사용자의 설정(언어 등)을 완벽하게 초기화
3. 보안: 관리자 권한이 필요한 작업을 원격에서 안전하게 수행

Client 주요 소스 코드 - 시스템 자원 수집

```
def collect_dynamic_info():
    # 1. CPU 및 메모리 사용량 수집
    cpu_usage = psutil.cpu_percent(interval=1)
    ram = psutil.virtual_memory()

    # 2. 실행 중인 프로세스 필터링
    processes = []
    for proc in psutil.process_iter(['name']):
        if proc.info['name'] not in WINDOWS_SYSTEM_PROCESSES:
            processes.append(proc.info['name'])

    # 3. 데이터 구조화 및 반환
    return {
        "cpu_usage": cpu_usage,
        "ram_used": round(ram.used / (1024**3), 2),
        "processes": json.dumps(processes)
    }
```

- 사용 파일: client/collector.py
  1. 자원 모니터링: psutil 라이브러리를 통해 CPU, RAM, Disk 정보를 실시간 수집
  2. 데이터 정제: 수많은 윈도우 시스템 프로세스를 필터링하여, 관리자가 꼭 봐야 할 사용자 프로세스만 추출
  3. JSON 통신: 수집된 데이터를 JSON 형태로 가공하여 서버로 전송

Server 주요 소스 코드 - 실시간 명령 배포

```
@app.route('/api/client/command')
def api_client_command():
    # ... (클라이언트 식별 로직) ...
    start_time = time.time()
    # 설정된 타임아웃 시간 동안 연결 유지 (Long-polling)
    while time.time() - start_time < timeout:
        # 1. 대기 중인 명령(pending) 조회
        cmd = db.execute(
            'SELECT * FROM pc_command WHERE pc_id = ? AND status = ?',
            (pc['id'], 'pending')
        ).fetchone()
        # 2. 명령이 있으면 즉시 반환
        if cmd:
            db.execute('UPDATE pc_command SET status=? ...', ('executing',
...))
            return jsonify({
                'command_type': cmd['command_type'],
                'command_data': cmd['command_data']
            })
        time.sleep(0.5) # 0.5초 간격으로 DB 재확인

    return jsonify({'command_id': None}) # 명령 없음
```

- 사용 파일: server/app.py
  1. 지연 응답: 클라이언트의 요청을 일정 시간(예: 30초) 동안 유지하며 대기
  2. DB 조회: 0.5초 간격으로 데이터베이스의 명령 큐(Queue)를 확인
  3. 즉시성: 관리자가 명령을 내리는 순간, 대기 중이던 루프가 명령을 감지하고 클라이언트에 즉시 전송

Server 주요 소스 코드 - 대규모 PC 일괄 제어

```python
@app.route('/api/pcs/bulk-command', methods=['POST'])
def api_bulk_command():
    data = request.json
    pc_ids = data.get('pc_ids', []) # 선택된 PC ID 목록

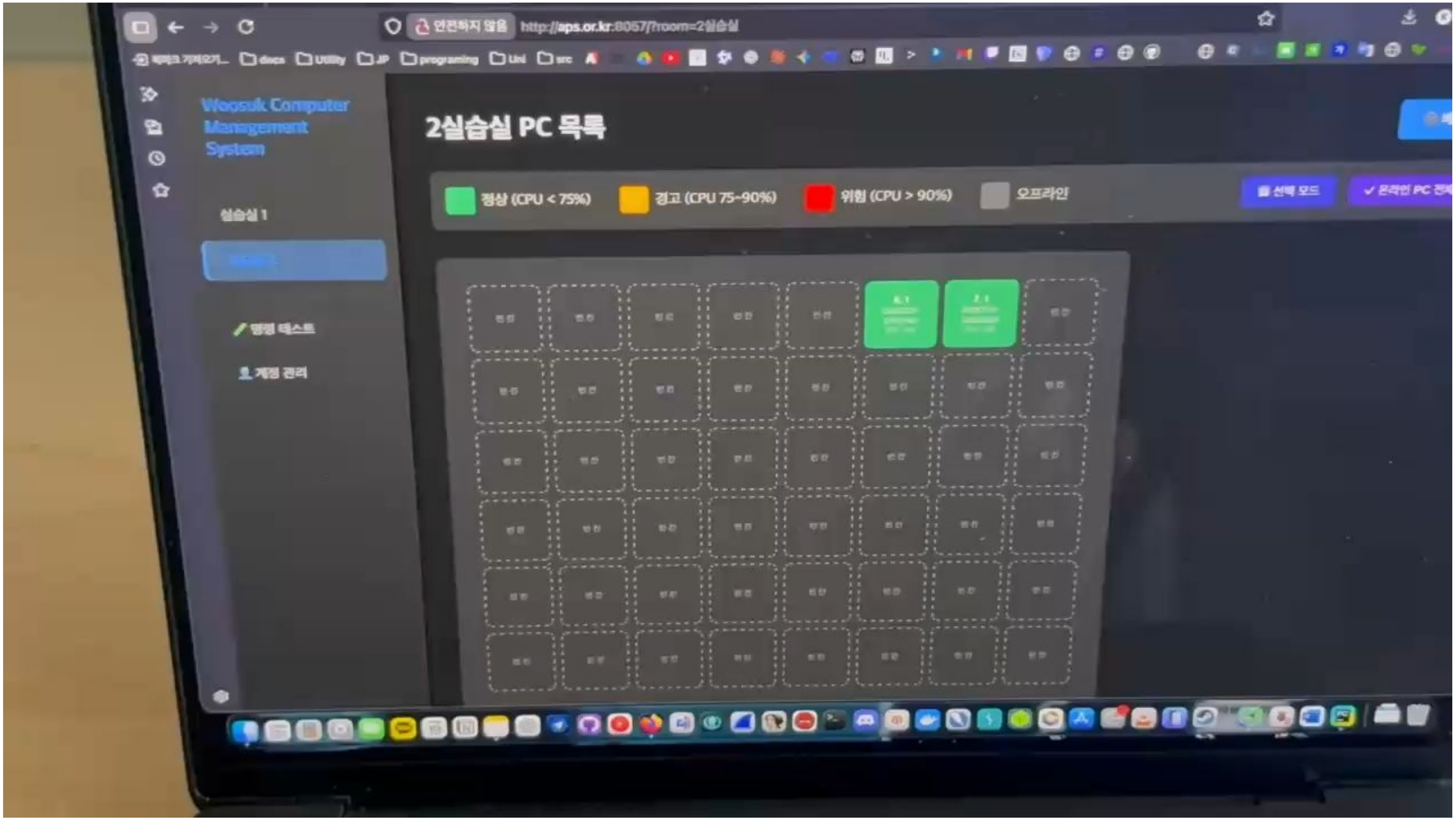
    # 1. 선택된 모든 PC에 대해 반복
    for pc_id in pc_ids:
        try:
            # 2. 각 PC별 명령을 DB 큐에 삽입
            db.execute('''
                INSERT INTO pc_command (pc_id, command_type, status)
                VALUES (?, ?, 'pending')
            ''', (pc_id, data['command_type']))

            results.append({'pc_id': pc_id, 'status': 'success'})
        except Exception as e:
            results.append({'pc_id': pc_id, 'status': 'error'})

    db.commit() # 3. 일괄 저장 (트랜잭션)
    return jsonify({'results': results})
```

- 사용 파일: server/app.py
  1. 다중 처리: 웹 UI에서 선택한 다수의 PC ID를 배열로 수신
  2. 명령 큐잉: 반복문을 통해 수십 대의 PC에 해당하는 명령을 데이터베이스에 순차적으로 적재
  3. 확장성: PC 대수가 늘어나도 안정적으로 명령을 배포할 수 있는 구조

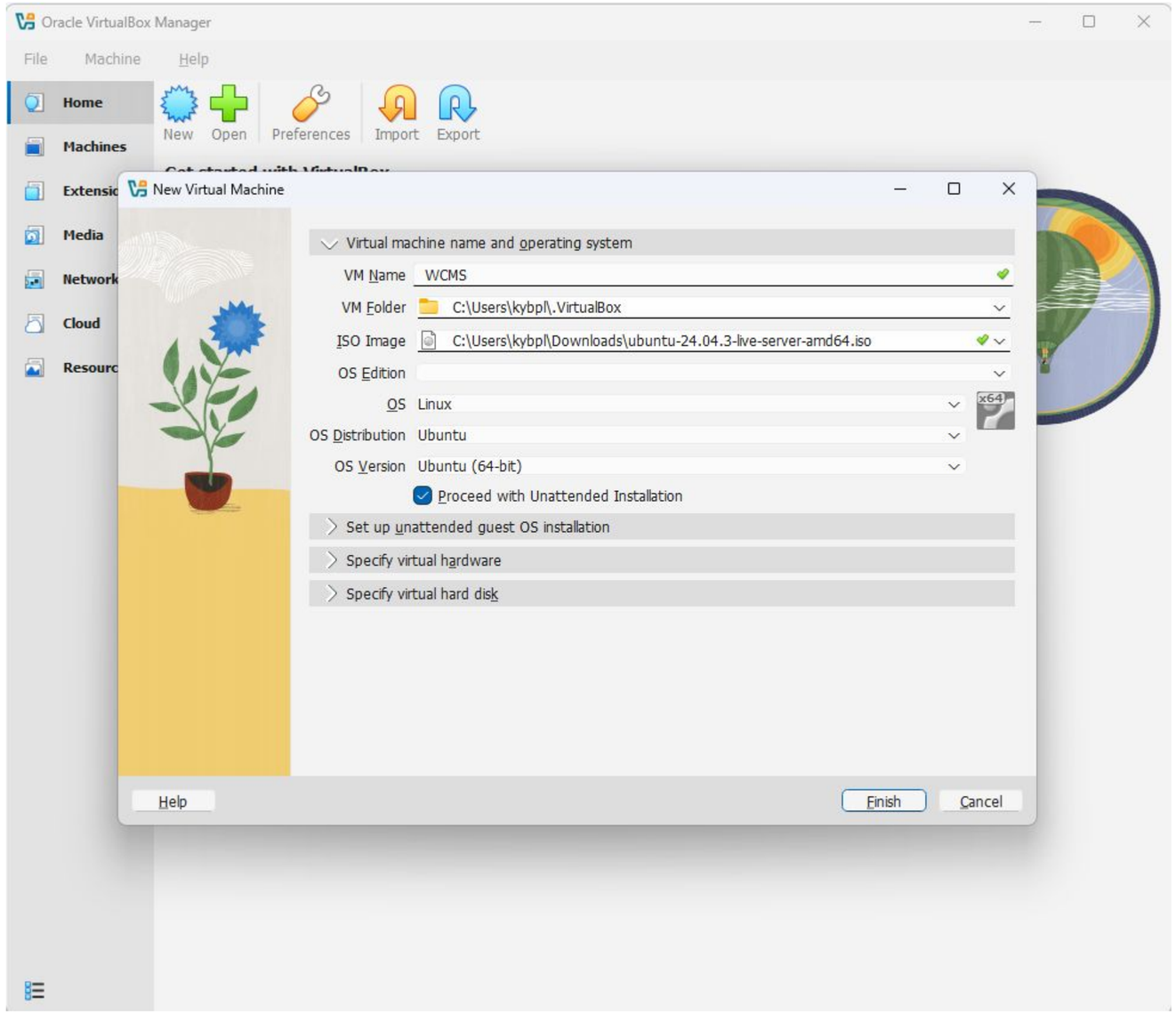
시연 동영상


안전하지 않음
http://aps.or.kr:8067/?room=2실습실
Woosuk Computer Management System
2실습실 PC 목록
정상 (CPU < 75%)
경고 (CPU 75~90%)
위험 (CPU > 90%)
오프라인
실습실 1
명령 테스트
계정 관리

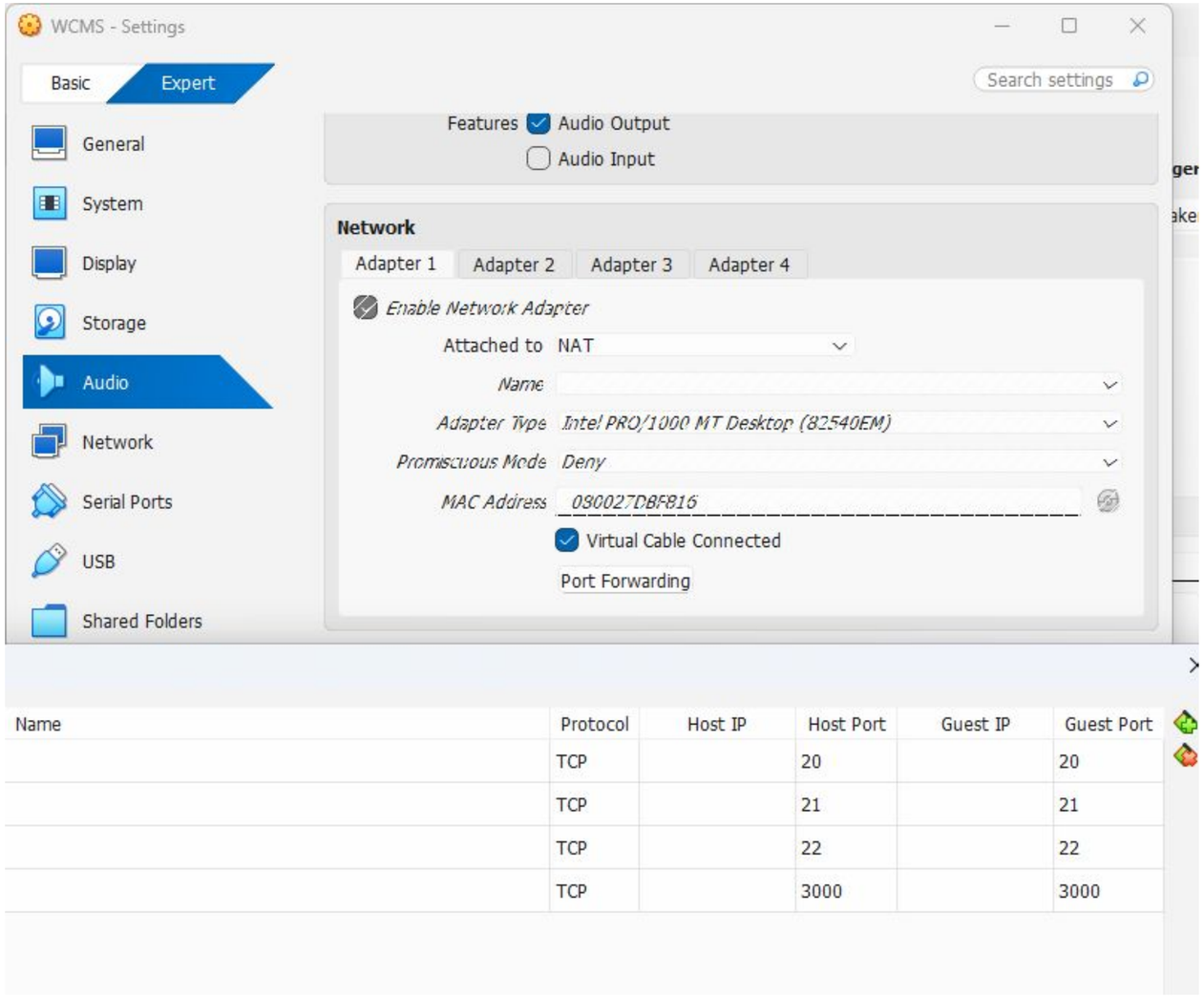
3. 실행방법

환경설정 - Server

- Oracle VirtualBox 7.2.4
- OS: ubuntu-24.04
- Memory: 4096MB
- CPU: 1
- Disk Size: 30GB
- Network: Nat
- 계정: master



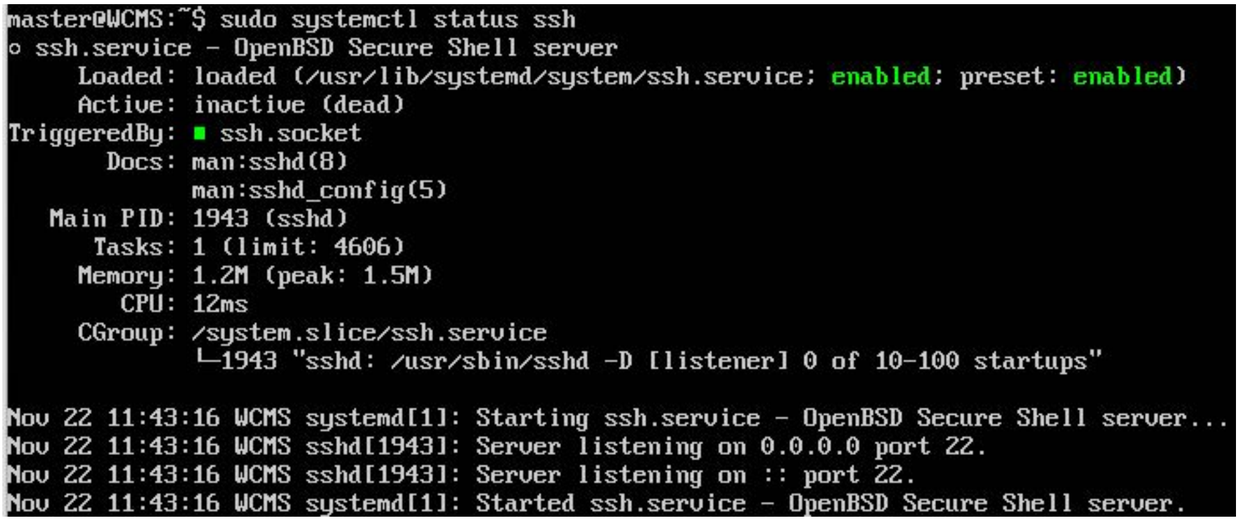
환경설정 - Server



서버 환경설정은 ssh를 사용해서 설정을 진행하겠습니다.

20, 21,22,3000번 포트포워딩 해주세요.

환경설정 - Server



ssh 설정 확인

> sudo apt update

> sudo apt install openssh-server

> sudo systemctl start ssh

> sudo systemctl enable ssh

> sudo systemctl status ssh

환경설정 - Server

```
PS C:\Users\kybpl> ssh master@localhost
The authenticity of host 'localhost (127.0.0.1)' can't be established.
ED25519 key fingerprint is SHA256:1ej2RH7YYmQNF3hv6tR/NyDjVbgcsqWiWHTevThQiOs.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
master@localhost's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-88-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat Nov 22 12:01:43 PM UTC 2025

  System load:             0.03
  Usage of /:              8.9% of 29.36GB
  Memory usage:            6%
  Swap usage:              0%
  Processes:               101
  Users logged in:         1
  IPv4 address for enp0s3: 10.0.2.15
  IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fedb:f816


Expanded Security Maintenance for Applications is not enabled.

30 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


master@WCMS:~$
```

이제 윈도우 터미널을 띄우고 다음 명령어를 타이핑 하면 됩니다.

> ssh master@localhost

ssh 접속 확인.

환경설정 - Server

```
(server) master@WCMS:~/WCMS/server$ ./init_db.sh
wal
migrations/schema.sql 기반으로 db.sqlite3 생성 완료.
(server) master@WCMS:~/WCMS/server$ python3 create_admin.py admin admin
✅ 'admin' 계정이 생성되었습니다.

계정 정보:
  - 사용자명: admin
  - 비밀번호: admin

⚠️  프로덕션 환경에서는 반드시 비밀번호를 변경하세요!
(server) master@WCMS:~/WCMS/server$
```

```
(server) master@WCMS:~/WCMS/server$ nohup gunicorn -w 2 -b 0.0.0.0:3000 app:app &
[1] 1465
(server) master@WCMS:~/WCMS/server$ nohup: ignoring input and appending output to '
nohup.out'
```

git clone --depth=1 https://github.com/Nekonic/WCMS.git
cd WCMS/server/

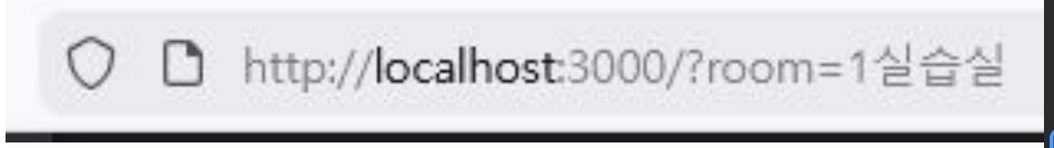
sudo apt install python3-venv sqlite
python3 -m venv server
source server/bin/activate
pip install -r requirements.txt

./init_db.sh
python3 create_admin.py 아이디 '비밀번호'
python3 create_admin.py admin admin

pip install gunicorn
grep -c processor /proc/cpuinfo
nohup gunicorn -w 2 -b 0.0.0.0:3000 app:app &

nohup gunicorn -c gunicorn_config.py app:app > /dev/null 2>&1 &
아래는 구니콘 프로세스 찾기, 삭제
ps -ef | grep gunicorn
pkill gunicorn

환경설정 - Server

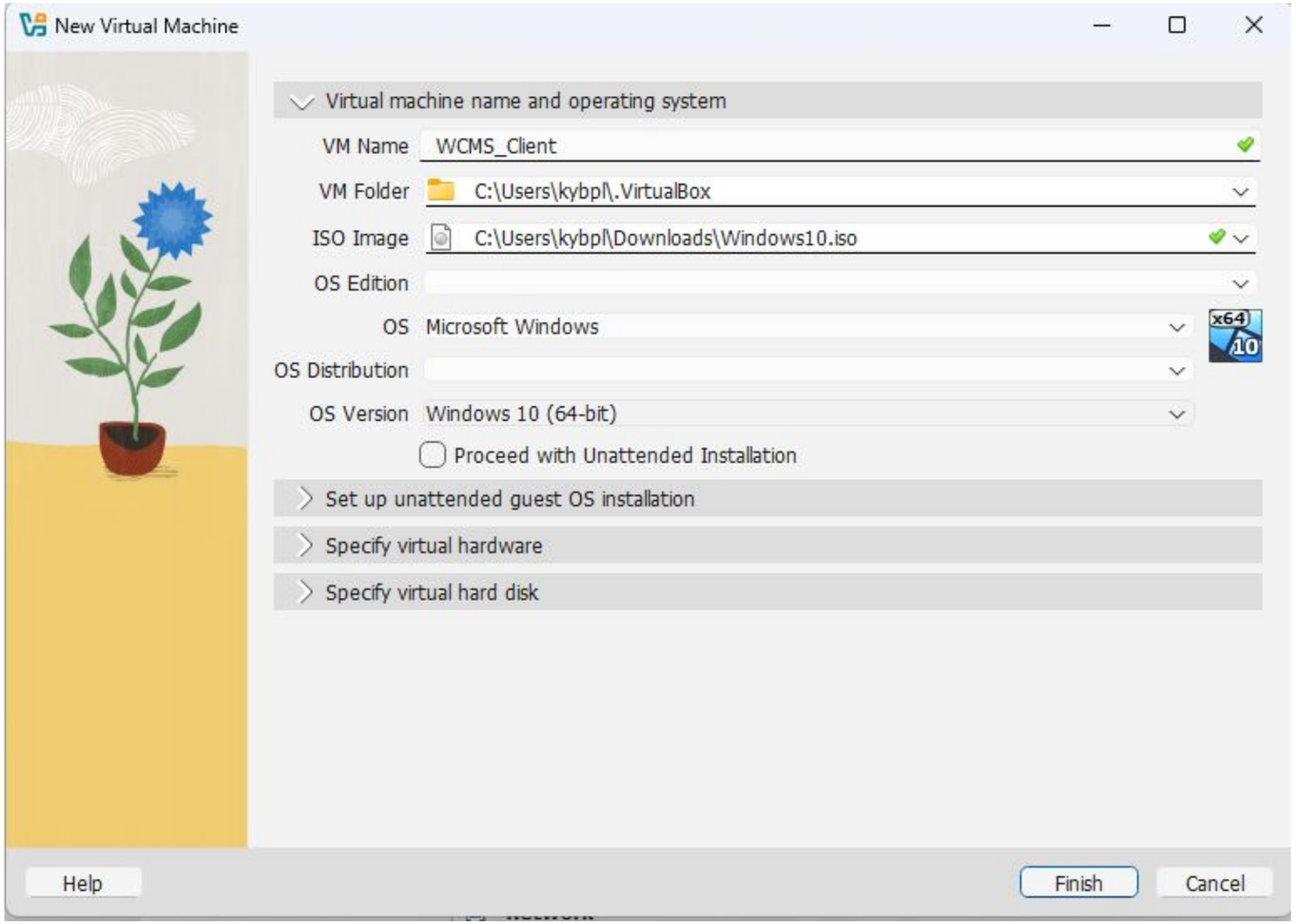


본체에서 localhost:3000 들어가서, 화면이 잘 나오는지 확인.



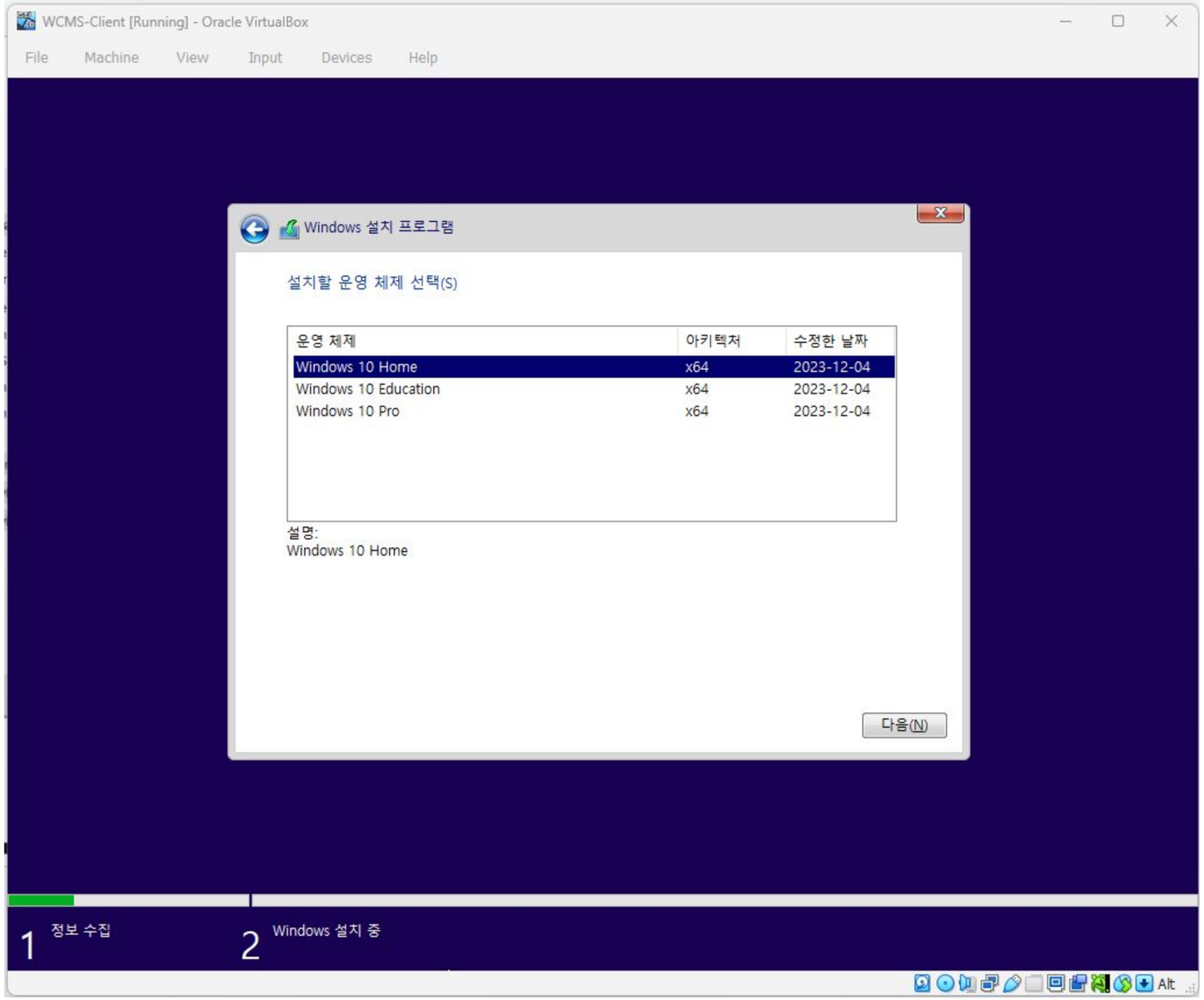
환경설정 - Client

- Oracle VirtualBox 7.2.4
- OS: windows10 22H2 64bit
- Memory: 4096MB
- CPU: 1
- Disk Size: 40GB
- Network: Bridged Adapter



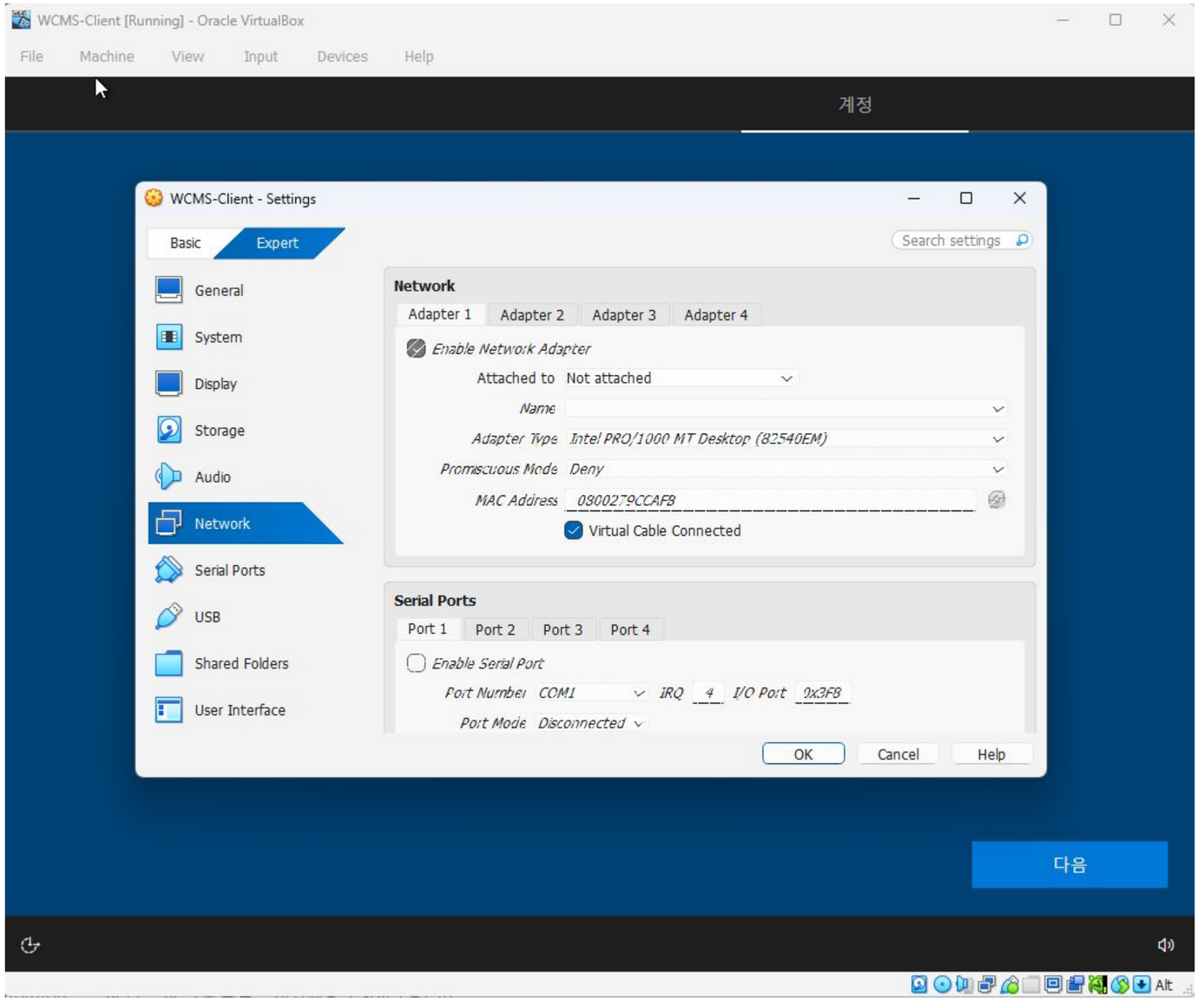
환경설정 - Client

기능이 가장 적은 Home 버전을 설치해줍니다.



환경설정 - Client

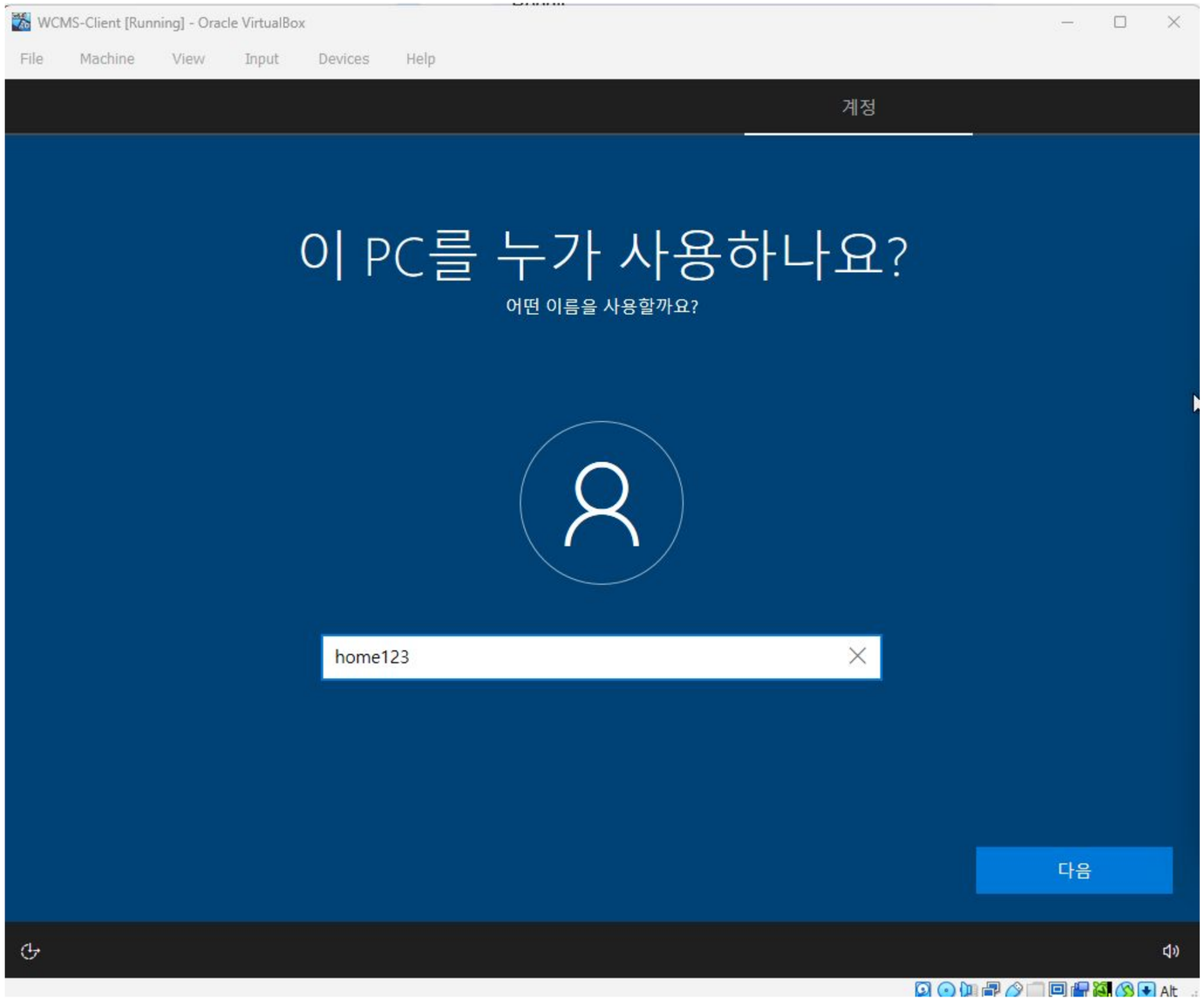
계정 설정을 위해서 잠시 네트워크 설정을 화면과 같이 해줍니다.



환경설정 - Client

이름은 home123 으로 진행
하겠습니다.

비밀번호까지 설정을 다
하셨다면, 네트워크를 원래대로
Bridged Adapter로
바꿔주세요.



환경설정 - Client

VM에서 서버가 들어가 지는지 확인해주세요.

아래는 VM이 아닌 본체입니다.

```
PS C:\Users\kybpl> ipconfig

Windows IP 구성


이더넷 어댑터 Hamachi:

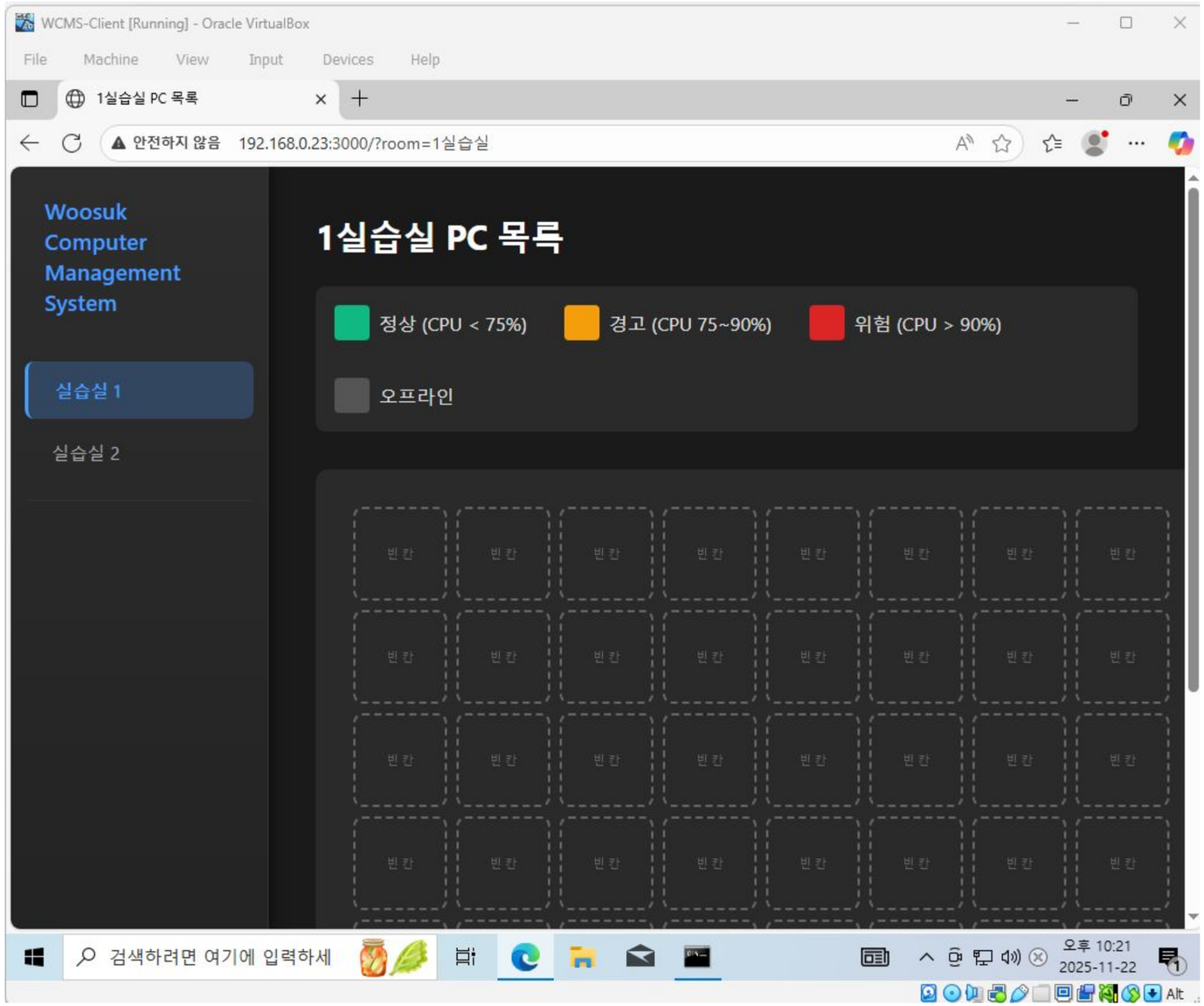
   연결별 DNS 접미사. . . . :
   IPv6 주소 . . . . . . . . . . . : 2620:9b::1905:dbf6
   링크-로컬 IPv6 주소 . . . . . : fe80::a173:5016:9546:ef74%8
   IPv4 주소 . . . . . . . . . . . : 25.5.219.246
   서브넷 마스크 . . . . . . . . : 255.0.0.0
   기본 게이트웨이 . . . . . . . : 2620:9b::1900:1

이더넷 어댑터 이더넷:

   연결별 DNS 접미사. . . . :
   링크-로컬 IPv6 주소 . . . . . : fe80::b89d:eb17:577a:d653%4
   IPv4 주소 . . . . . . . . . . . : 192.168.0.23
   서브넷 마스크 . . . . . . . . : 255.255.255.0
   기본 게이트웨이 . . . . . . . : 192.168.0.1

이더넷 어댑터 이더넷 2:

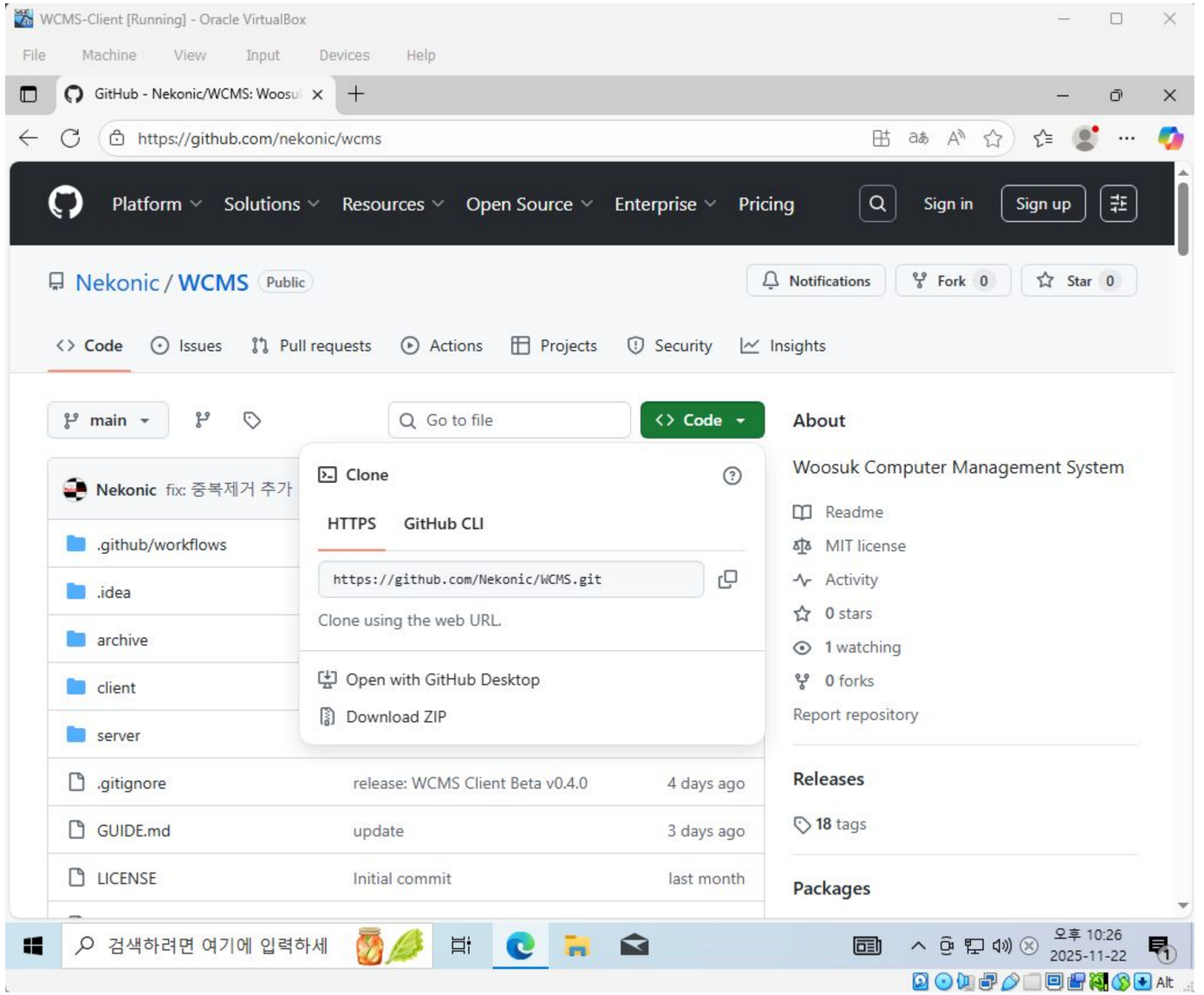
   연결별 DNS 접미사. . . . :
   링크-로컬 IPv6 주소 . . . . . : fe80::643a:5097:d06c:6c58%12
   IPv4 주소 . . . . . . . . . . . : 10.0.0.1
   서브넷 마스크 . . . . . . . . : 255.255.255.0
   기본 게이트웨이 . . . . . . . :
```



환경설정 - Client

https://github.com/Nekonic/WCMS

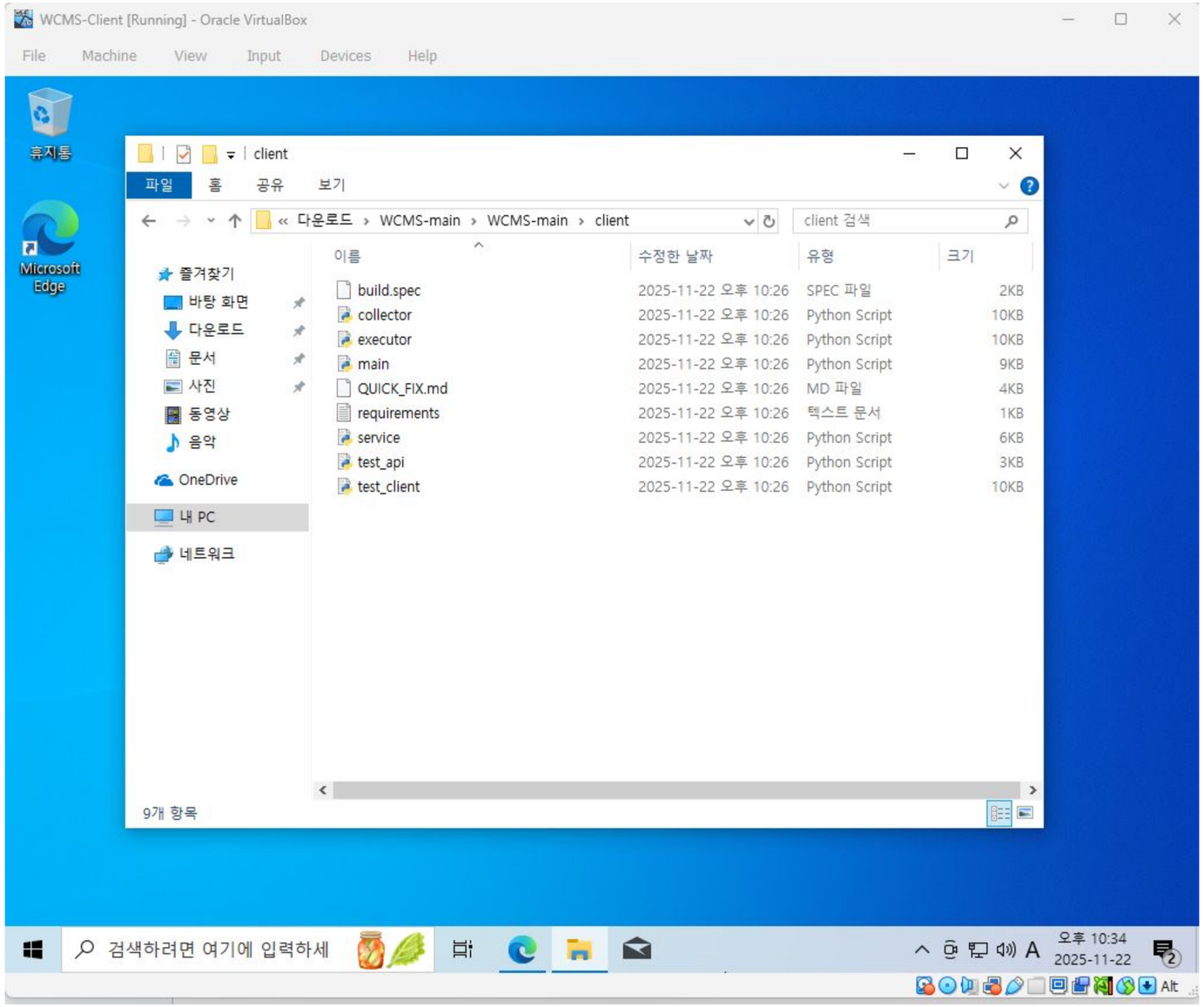
에 들어가서, Code -> Download ZIP 을 눌러서 다운로드 받습니다.



환경설정 - Client

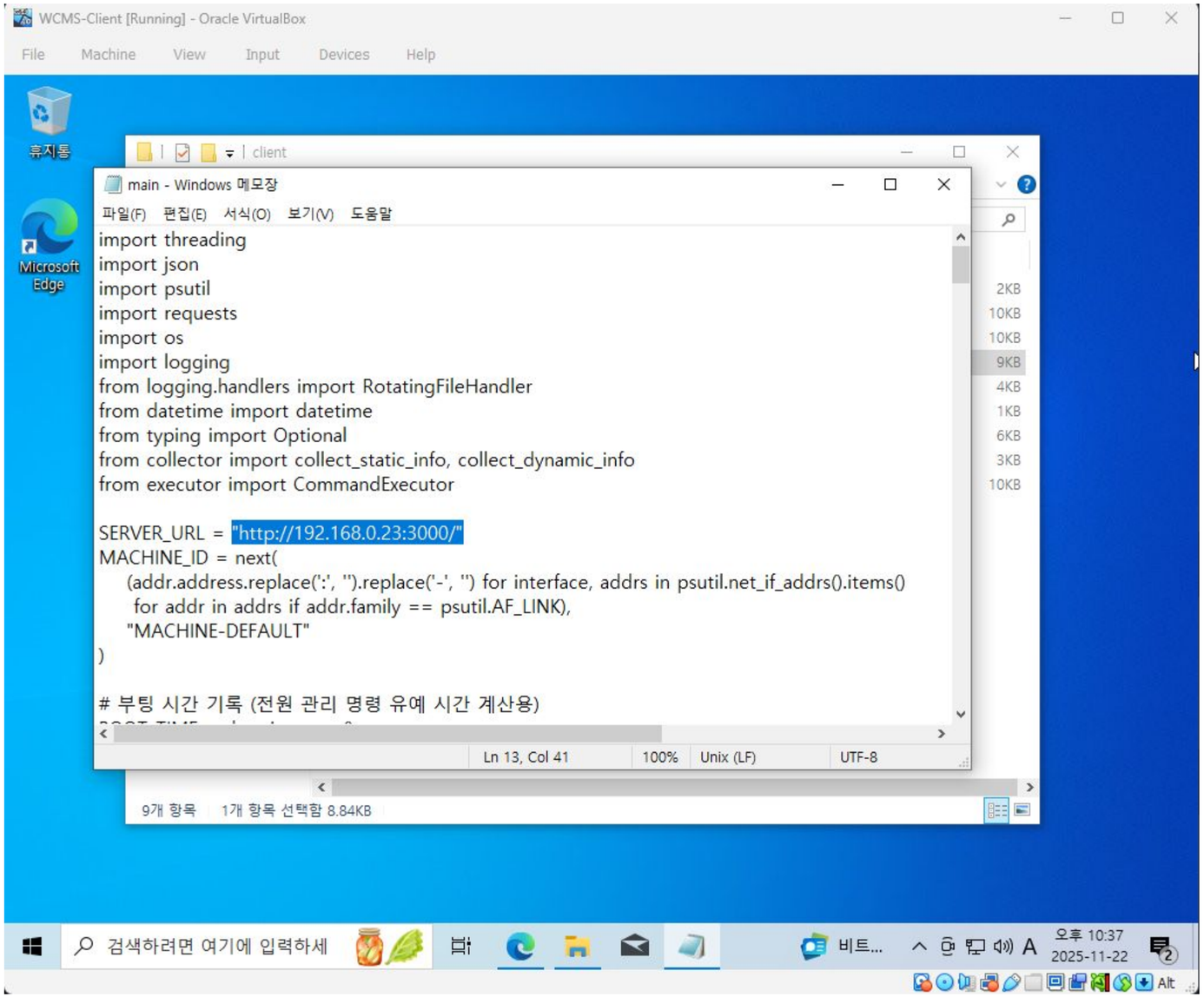
압축을 해제하고 파이썬을 설치해줍니다.

파이썬 버전은 최신버전을 설치해줬습니다.(3.14)



환경설정 - Client

main.py에 있는
SERVER_URL을 자신의 서버로
바꿔줍니다.



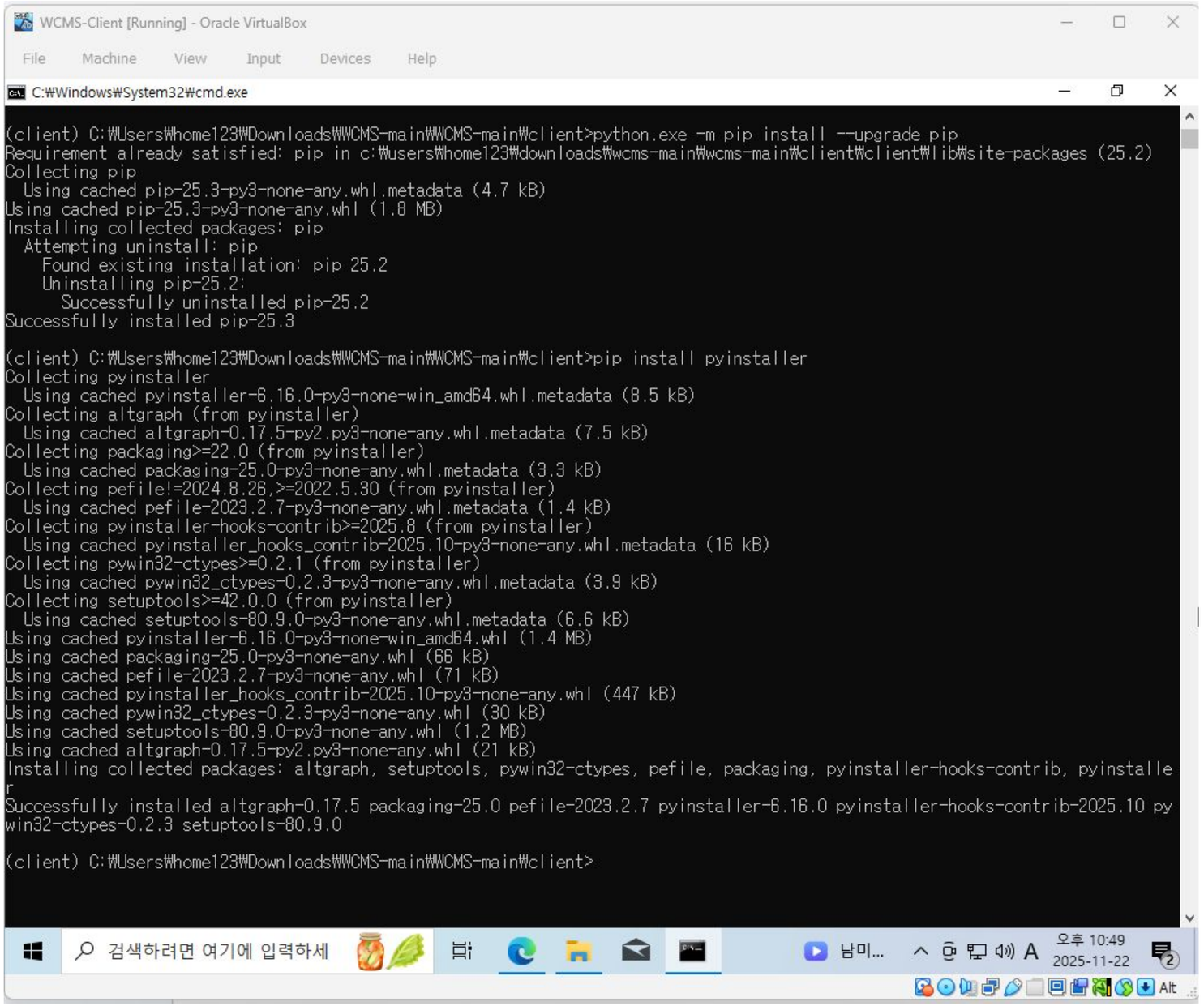
환경설정 - Client

python -m venv client

client\Scripts\activate

pip install pyinstaller

빌드에 필요한 라이브러리를 설치해줍니다.



환경설정 - Client

python -m venv client

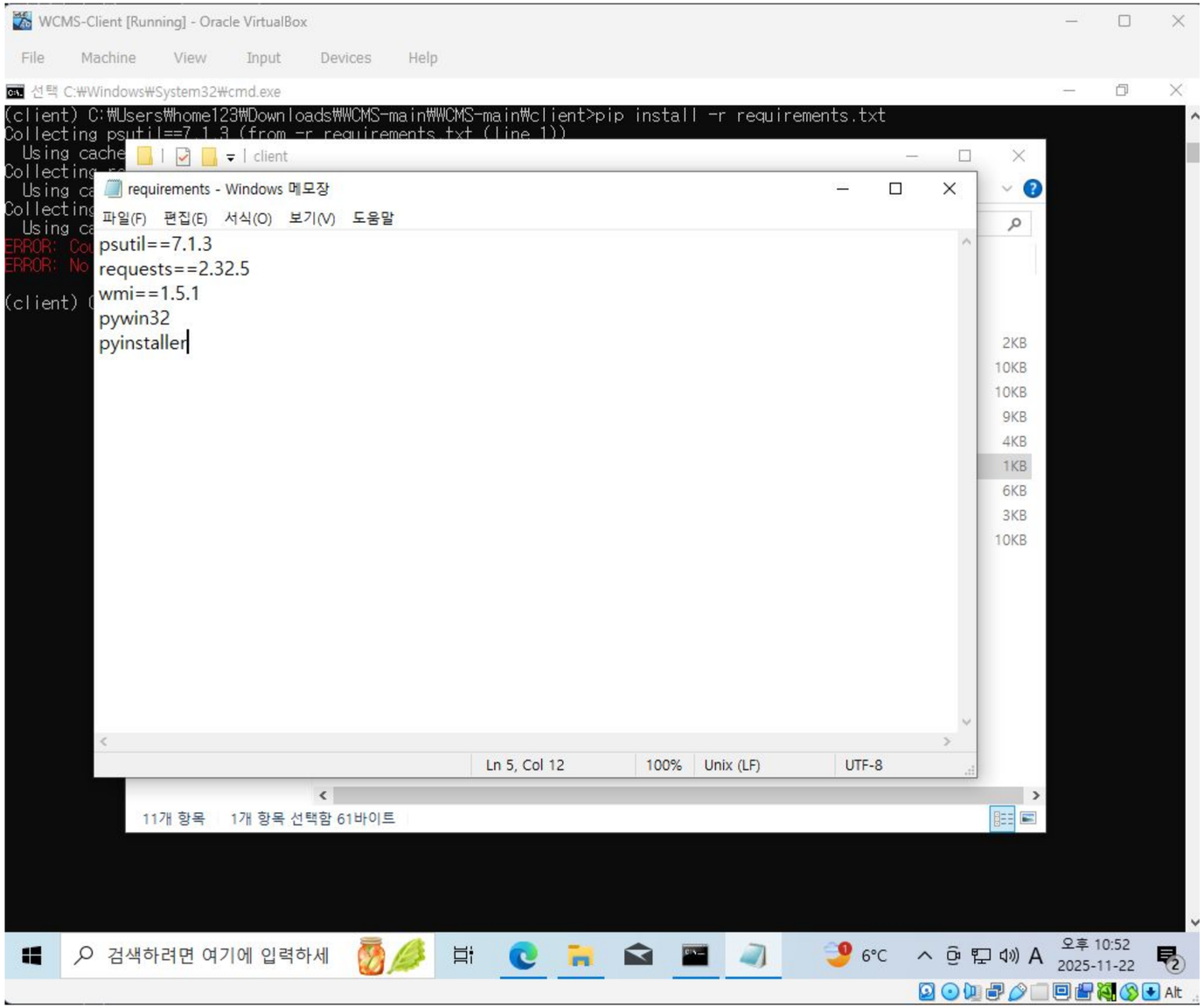
client\Scripts\activate

python -m pip install --upgrade pip

requirements.txt에서 pywin32 뒤에 “==306” 을 지워줍니다.

pip install -r requirements.txt

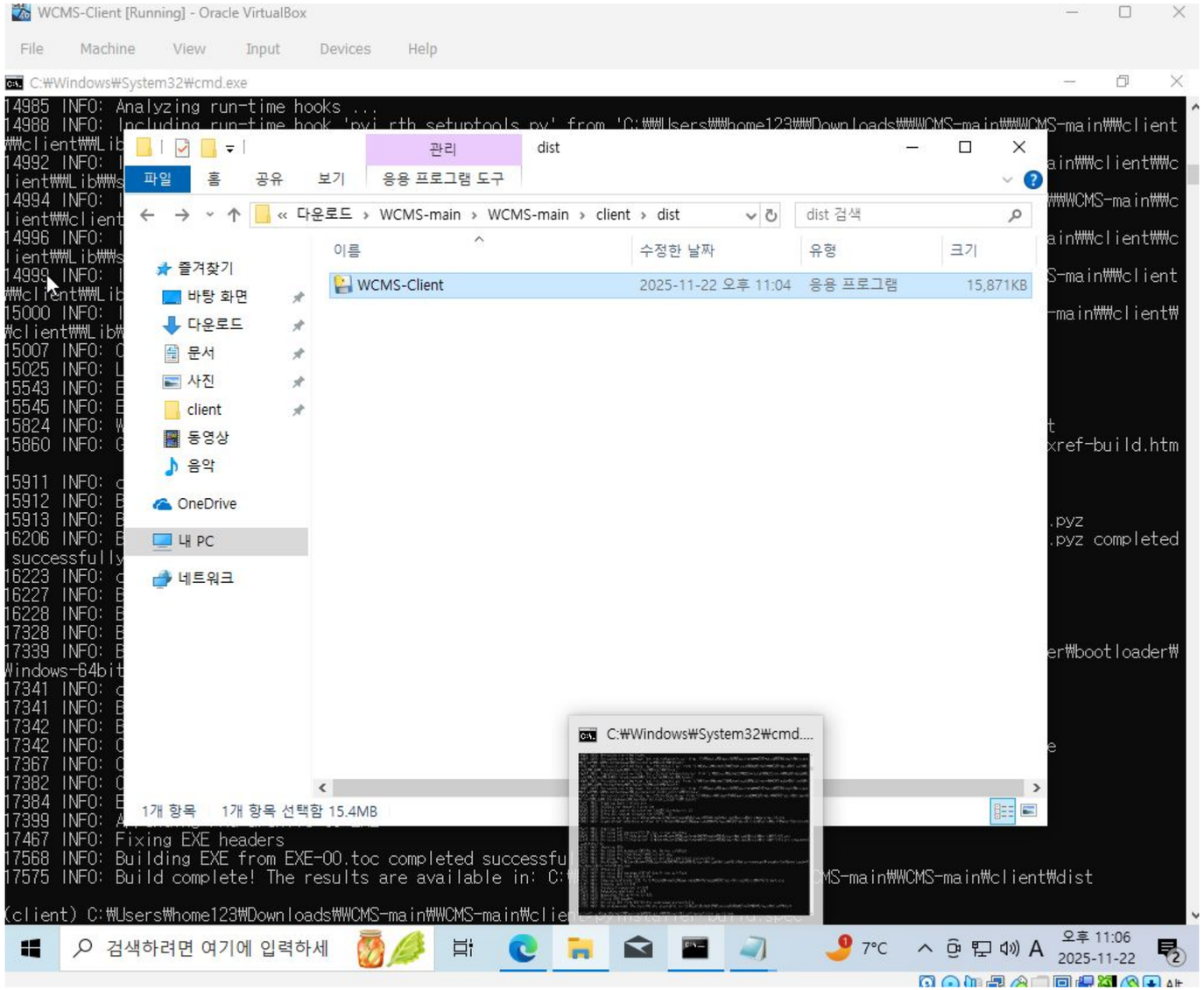
빌드에 필요한 라이브러리를 설치해줍니다.



환경설정 - Client

pyinstaller build.spec

정상적으로 빌드가 되었다면 화면과 같은 경로에 WCMS-Client가 생기게 됩니다.

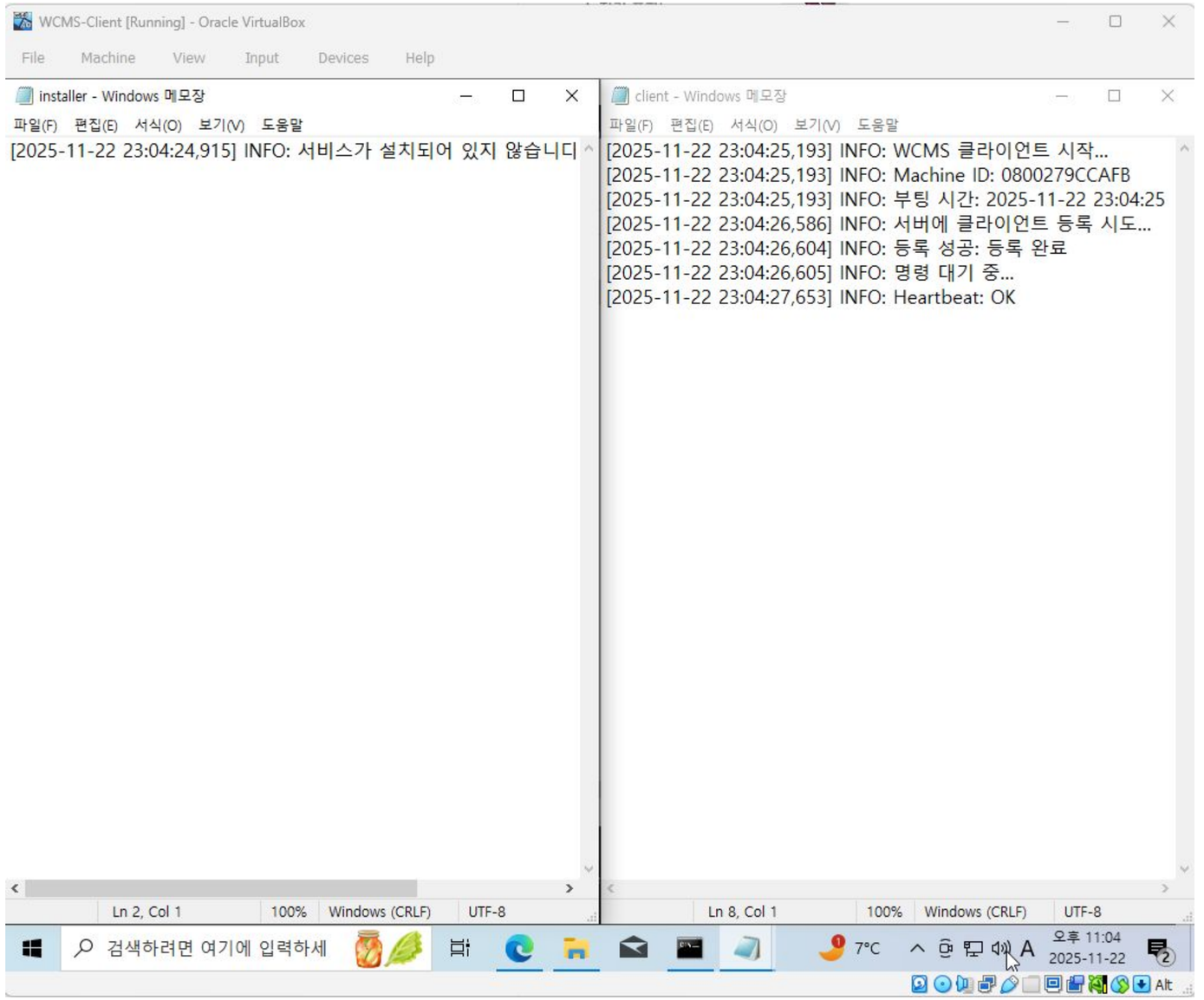


환경설정 - Client

관리자 권한으로 실행을 하고,

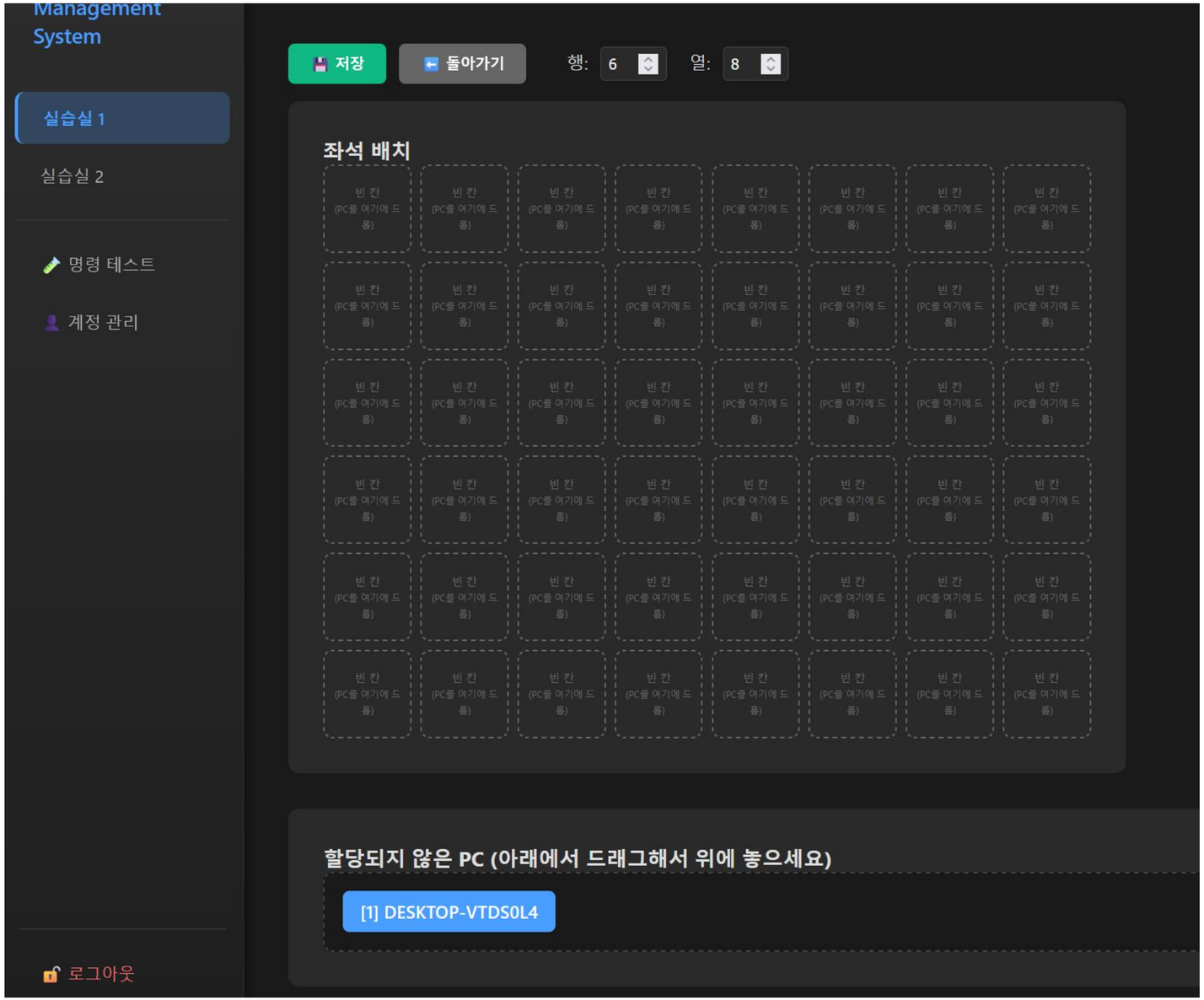
C:\ProgramData\WCMS\logs

이 경로에 들어가면 두개의 로그파일이 있는데, 내용은 화면과 같습니다.

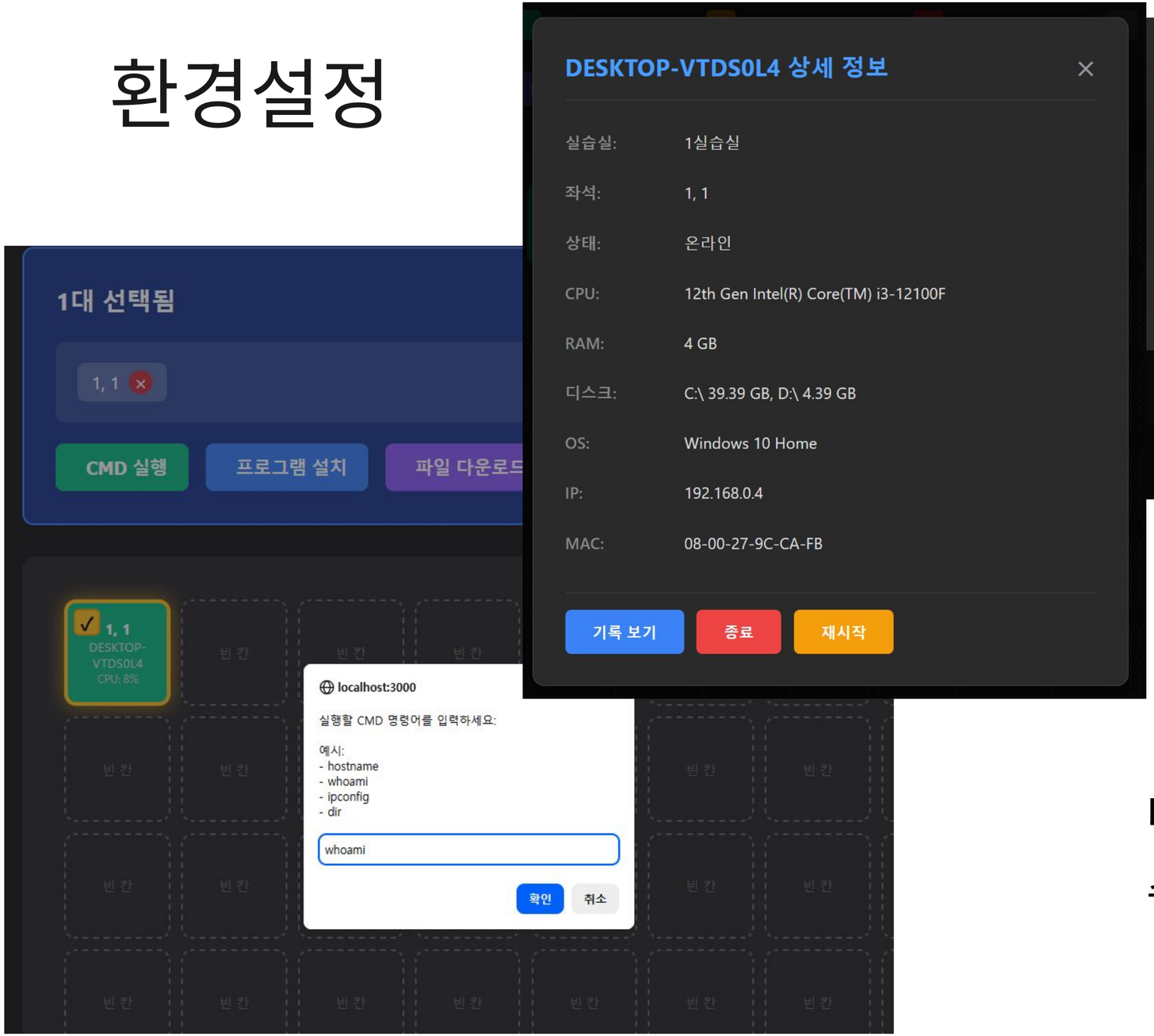


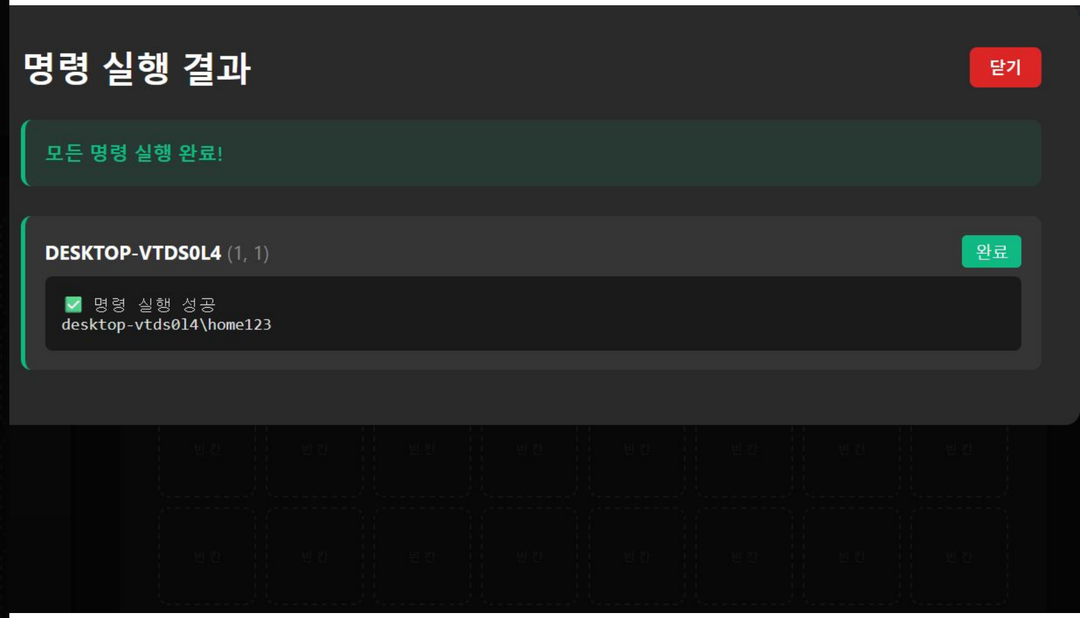
환경설정

이제, 로그인을 하고 배치편집을 누르면 다음과 같이 할당되지 않은 PC 한대가 보이면 성공입니다.



환경설정





다음과 같이 정상적으로 작동하는 것을 볼 수 있습니다.

계획

향후계획

1

실습실의 모든 PC에 설치하기,
디스크 사용량, 렘 사용량 등을 시각화 하기,
버그 수정하기

2

안정버전 1.0.0 만들기,
한국/몽골/중국 등의 국가별 계정 만들어서 언어설정 관리하기,
자동 업데이트기능 추가하기

3

다른 실습실에도 설치할 수 있도록 문서화,
정기적으로 보안점검하기,
서버, 클라이언트 코드 성능 개선하기

4

소프트웨어 유지보수를 위한 문서작성 및 테스트코드 작성하기,
서버 가용성 높이기 (로드 밸런서, 백업 등등)

알려진 버그

오프라인 PC의 상태가 온라인으로 되어있는 경우가 존재.

MpDefenderCoreService.exe 과 같이 수집에 필요없는 윈도우 기본 프로세스가 탐지됨.

참고자료

점프 투 플라스크 / 박응용

- https://wikidocs.net/book/4542

이성원 교수님

- http://woosuk.izerone.co.kr/manager/chfi2025

파이썬 해킹 레시피 / 공재웅 - 우석대학교 도서관

Thank you!

wie_weit@naver.com

https://github.com/Nekonic